

Ch-1 Introduction to Software Engineering

Software

► Program - a step by step instructions to perform a specific task on a computer in a programming lang is called a prg (a set of insts).

► Software - is a program/collection of integrated prgs with proper documentation (requirement analysis, design, coding, testing) & user manuals which mainly includes installation guide & other manuals.

$S/w = prg + documentation.$

► Software components

- Prg
- Documentation - S/w documentation consists all the description, prgs, graphics & insts. pertaining to design, coding, testing & prep. of S/w.
- Operating procedure (User manual & operational manual) provides info abt what S/w is, how to work with it, how to install it on the system, how to control all the activities of the S/w.

► Software Crisis

- ① Project delays & Cost overruns - many S/w prgs exceeds their initially planned completion deadline & estimated budget significantly.
- ② Poor quality & reliability - sometimes the S/w contains numerous bugs causing reliability issues as ~~the~~ it is prone to failures & difficult to use.

(iii) Maintenance Difficulties - maintaining & updating s/w ^{sometimes becomes} extremely challenging. Changes to s/w often introduces new bugs & primes.

(iv) Lack of Proper Management ^{skilled personal} & Processes - leads to inefficient development practices

(v) Failed to meet user needs & requirements - due to significant gap b/w what user needed & what developers delivered.

(vi) Rapidly changing technology - Outstripped the ability of software development practices to keep up.

Software Engineering

SE is the systematic application of engineering principles and methods to the design, development, testing and maintenance of software products.

It involves the use of various tools, techniques, procedures, methodologies & well defined scientific principles to manage the software development process and ensure the quality, reliability & maintainability of software products.

Characteristics of software

① Intangibility - unlike physical products, s/w is intangible & doesnot have a physical form to be seen, touched. They only exist as code & functions in computer/digital devices.

② Complexity - can be extremely complex, consisting of millions of lines of codes (LOC).

③ Changeability - s/w is malleable & can be easily changed or updated to fix bugs, new features & improve performance.

④ Dependability - s/w must be reliable, ensuring to perform

as intended with correctness, consistency and must be secure, reliable &.

- ⑤ Scalability - S/w must be able to handle increasing loads, whether it's more users, data, transaction, without much degradation in performance.
- ⑥ Maintainability - S/w must be easy to maintain & update with clear documentation, & standard modular design.
- ⑦ Efficiency - should use resources (CPU, mem, net. bandwidth) efficiently to ensure optimal performance.
- ⑧ Reusability - should be reusable in diff. contexts & applications.
- ⑨ Portability - should be able to run on diff platforms & environment with minimal modifications.
- ⑩ User-friendliness - should be intuitive & easy to use, ensuring a +ve user experience.
- ⑪ No physical wear & tear - does not deteriorate overtime like physical objects.

Why should we study SE?

- i Provides solid foundation in CS principles, programming & DS.
- ii Teaches critical thinking & problem solving skills.
- iii we acquire skills of design, interface dev, testing, project management, system analysis, teamwork.
- iv Understanding S/w Lifecycles.
- v Efficient ~~under~~ scalability & performance in case of large ^{datasets}.
- vi High demand, high paying job & versatile career opportunities.
- vii Cost Reduction needed.

# Program	Software
i) Step by step insts (sets) written in a prog lang to perform a specific task. ii) Usually focused on a single specific task. iii) Typically simpler & smaller in size. iv) Components include only source code. v) May or maynot have user interface, offer very little. vi) Easier maintenance. vii) May have minimal/no documentation. viii) Author himself or small grp of specific users are target audience. ix) Eg: a calculator app	i) Collection of prgs, data & related resources that work tog to perform multiple tasks. ii) Can encompass multiple prgs & offer broader functionality. iii) Often complex, with multiple integ-rated prgs & components. Larger size. iv) Components - prgs, libraries, documentation, user interface & more. v) Typically includes a user-friendly interface. vi) Requires ongoing maintenance & updates due to complexity. vii) Comes with extensive documentation, user manuals & help files. viii) Designed for large no of users with varying tech knowledge. ix) MS Office including Excel, word, powerpoint etc.

# Hardware	Software
i) Physical components of a system such as CPU, mouse etc. ii) Cannot be easily modified. Requires physical changes/replacements to update upgrade/modify. iii) Subject to phys. wear & tear, aging & degradation.	i) Intangible set of prgs & operating info used by a system that exists in form of code & data. ii) Can be easily modified through reprogramming & upgrades. iii) Not sub to wear & tear but can become obsolete over time.

(iv) High manuf. costs & logistic expenses.

(v) Physically stored as components or devices.

(vi) Executes inst provided by S/W to perform phys. operations.

(vii) Developed through design, prototyping & manuf. process.

(viii) User interactions through i/o devices.

(ix) Errors require phy. repair/ replacement of components

(i) Less exp. to distribute & replicate after initial dev.

(v) Stored electronically in mem, cloud storage or hard drive

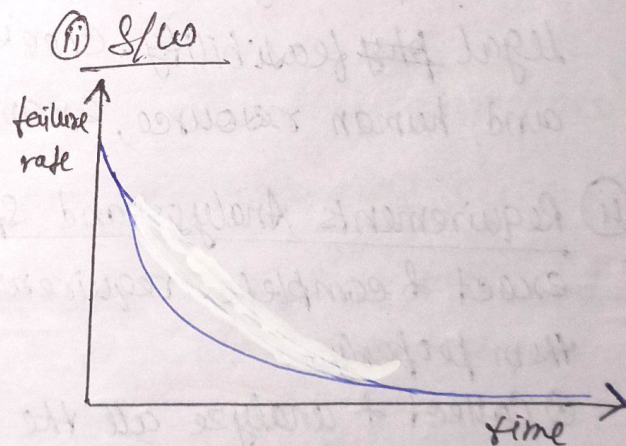
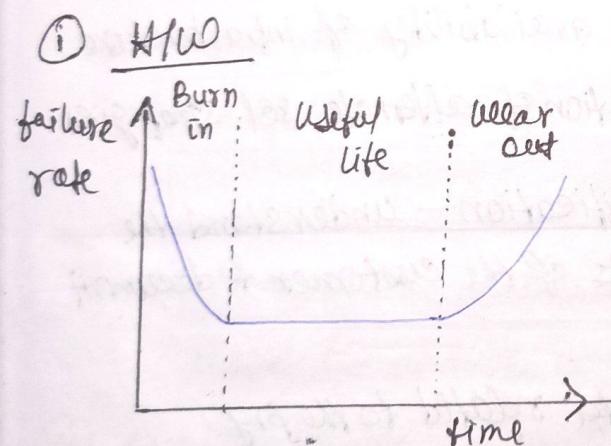
(vi) Provides inst & functionality to h/w to perform tasks.

(vii) Developed through coding, testing & debugging prog.

(viii) User interactions through GUIs & command line interfaces.

(ix) Errors require updates/patches

Failure Rates of Software & Hardware.



H/W/S/W cost



Principles of SE

- (i) Abstraction - involves hiding complex implementation details of a system & exposing only necessary & relevant parts. This helps in reducing complexity & focusing on high-level operations.
- (ii) Decomposition - involves breaking down a S/W system into distinct sections, each addressing a separate concern/aspect. Improves modularity & makes the system easier to manage.
- (iii) Encapsulation - involves bundling the data & methods that operate on the data into a single unit.

Phases of Software Development Life Cycles (SDLC)

- (i) Feasibility Analysis - determines whether the proj is viable & worth pursuing. Assess technical, economic, operational, legal ~~proj~~ feasibility. Checking availability of infrastructure and human resource, examination of alternate sol. strategies.
- (ii) Requirements Analysis and Specification - understand the exact & complete requirements of the customer & document them properly.
 - ① Collect & analyze all the data related to the proj.
 - ② Document the requirement in detail along with the functionalities of the system, called Software Requirements Specification (SRS).
 - ③ Ensure validity of each requirement as a minor error can lead to severe prob in later stages.
- (iii) Design - we transform the requirements ~~for the~~ into a blueprint structure that is suitable for implementation of the code in a specific programming lang.

① System Design - define overall architecture - system's structure, components, & their interactions.

② High Level Design - create architectural designs - data models, system interface & user interface.

③ Low Level Design - detail the internal logic, algo & ds for each component.

④ Documentation - called Software Design Description (SDD), describing how the system will perform functions.

⑤ Coding - converting the design into executable code & integrate all components to build the software system.

⑥ Combine individual components in a unified system.

⑦ We create code documentation & user guides as needed.

⑧ Testing - we need to test & verify the correct, functionality of the software, free from any bugs, errors & faults that can lead to failure.

① Unit testing - test individual components.

② Integration testing - test interaction b/w ^{integrated} components.

③ System Testing - test complete integrated system.

④ Acceptance Testing - validate the system with end-users to ensure it meets their needs & expectation.

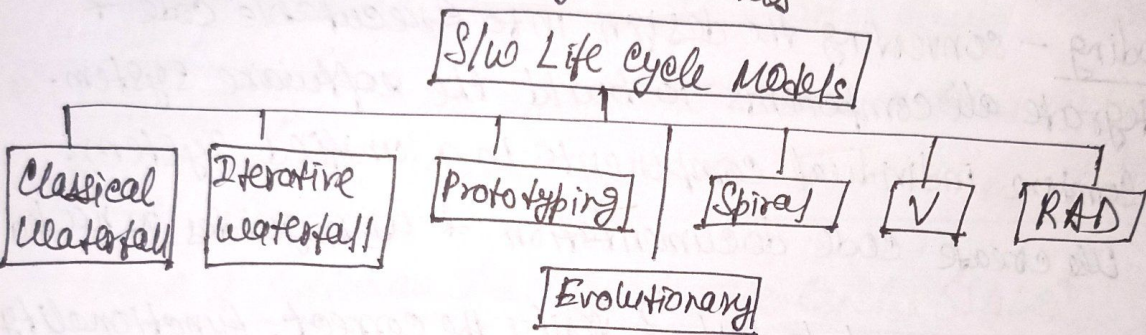
⑨ Maintenance - provide ongoing support & enhancements to ensure the s/w continues to meet user's ^{needs} & remains functional overtime. It includes - bug fixes, updates, performance monitoring.

Life Cycle Model

A S/w LCM is defined as —

- ① a descriptive & diagrammatic model of S/w Lc.
- ① identifies all activities required for prg dev.
- ① establishes a proper precedence order among diff activities
- ① divides Lc into diff phases.

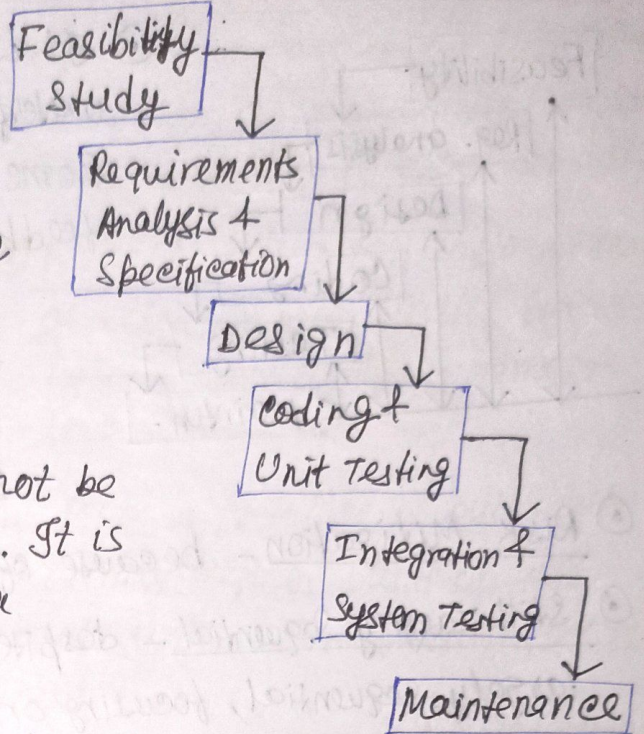
There are several life cycle models —



- ① The development team must identify a suitable LCM based on the type of S/w & always adhere to it.
- ① It helps in systematic & disciplined development of the S/w with proper entry & exit criteria for every phase.
- ① ~~while~~ During the development of the ~~prg~~ S/w, the S/w dev team often faces 99% complete syndrome. It refers to a situation where a prg appears to be nearly finished, but the few last remaining tasks take much longer than anticipated causing delays & frustration.

① Classical Waterfall Model

② It is one of the earliest & most traditional S/W dev methodologies that follows a linear & sequential approach, where each step relies on the completion of prev one.



③ Not a prac. model as it cannot be used in actual S/W dev prjs. It is generally considered to be a theoretical way of S/W dev.

④ All other LCM are essentially derived from it.

⑤ From Feasibility to Int. & System test is called development phase.

⑥ Max effort - in dev. phase is testing.

⑦ max effort - on whole is Maintenance

⑧ Called Waterfall model - as its diagrammatic repr. represents as of a waterfall.

⑨ Type of prjs

- Small to med sized prjs with clear well defined requirements
- Prjs with min changes during dev, + ^{more} predictable
- Low Risk proj.

⑩ No iteration - no going back to prev phases once completed.

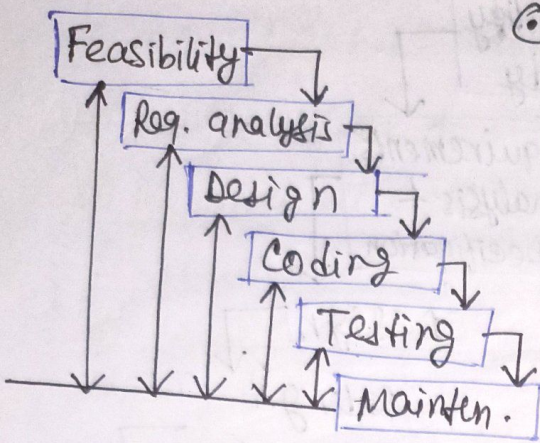
⑪ Advantages

- Simple & easy to understand
- Structured approach
- Clear doc. with well defined i/o.
- Well suited for low & fixed price contracts

⑫ Disadvantages

- Inflexibility
- High Risk & uncertainty
- Limited Customer feedback
- Not ideal for complex & uncertain prjs
- ^{med.} high cost
- High dev time.

② Iterative Waterfall Model



① It is a variation of the traditional waterfall model that incorporates some elements of iteration & feedback into the dev. process. While it still follows the sequential approach, but allows for revisiting & refining phases based on feedback/new info.

- ① Risk Mitigation - because of iteration & feedback.
- ① Still Largely Sequential - despite iter & febk, the model ^{is} still largely sequential, focusing on completing 1 phase before entering next one.
- ① Intermediate Deliverables - each stage produces deliverables that can be reviewed & refined before moving on to next phase.
- ① Advantages

- | | |
|----------------------------------|--|
| → Improved Flexibility | → Early error detection |
| → Structured & disciplined appo. | → More reliable & higher quality of final product. |
| → Better Risk management | → Practically Feasible |

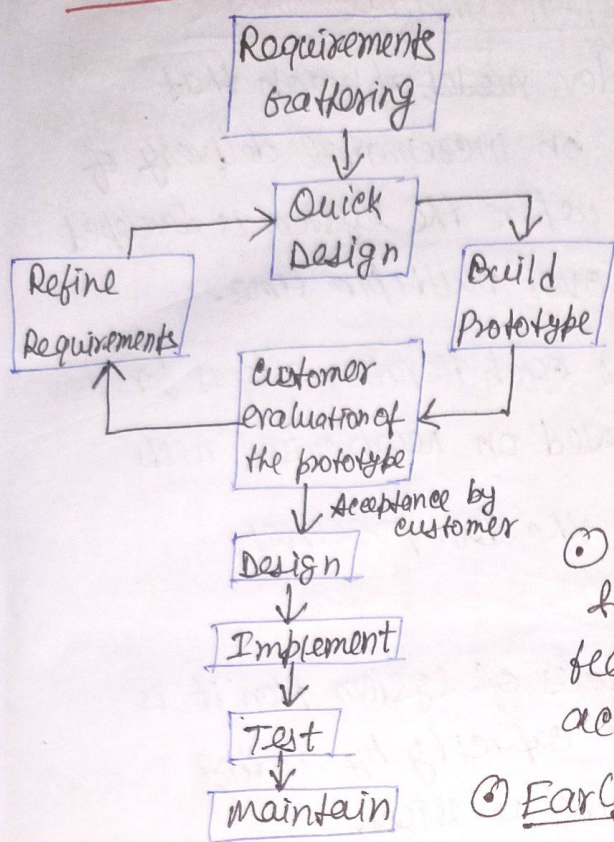
① Disadvantages

- | | |
|-----------------------------------|---|
| → Limited no of iteration | → Visiting earlier stages leads to delays |
| → Managing etc. & febk is complex | → Risk of over iteration |
| → Large T _{dev.} | |

① Types of proj

- | | |
|---|--|
| → med size proj | → Projs where most requirements are stable |
| → Projs with thorough documentation of each feedback. | |

③ Prototyping Model



① Most of the customer are not sure abt the functionality they require from the s/w, as a result the final s/w is not according to exact demand.

② Prototype - is a key implementation of a system with limited functionalities, inefficient performance, & low reliability.

③ In PTM, a prototype is built, tested & iteratively refined based on user feedback until a satisfactory sol. is achieved.

④ Early user involvement - to provide feedback

⑤ Quick & Incomplete versions, focusing on only key features & func.

⑥ Exploratory & Experimental - allowing developers to try diff ideas & approaches to find the best sol.

⑦ Focuses on refining & clarifying requirements.

⑧ Disposal or Evolutionary prototypes.

⑨ Advantages

- Better requi. understanding
- Reduced Risk
- Improved Communication
- More likely to meet user expectation due to feedback nature
- Flexibility & Adaptability

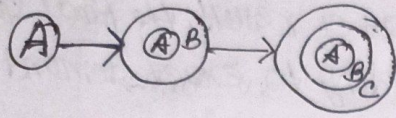
⑩ Disadvantages

- High Tdev & Cost dev
- High complexity
- Due to quick builds, prototypes ~~are~~ ^{Lack perf.} structure, scalability.
- If user is not satisfied with ~~the~~ ^{proto.} he may lose interest in the proj

⑪ Types of proj.

- Unclear & evolving requi.
- Proj with high risk & uncn.
- User centered proj
- Research & dev. proj.

④ Evolutionary / Successive Versions / Incremental Model (with iteration)



① It is a ^{s/w} dev. ~~model~~ approach that emphasises on incremental delivery of a system, where the system is developed & refined over multiple time.

② Flexibility in Requirements - as each iteration allows for adjustment & enhancements based on new/revised needs.

③ User involvement, throughout the dev. processes.

④ Less risks & uncertainty

⑤ First we develop the core modules of system then it is refined into increasing lvs of capacity by adding new functionalities in successive version.

⑥ Advantages

- | | |
|--------------------|--------------------------------|
| → Flexibility | → Improved risks |
| → User involvement | → Enhance quality. |
| | → useful for larger size prjms |

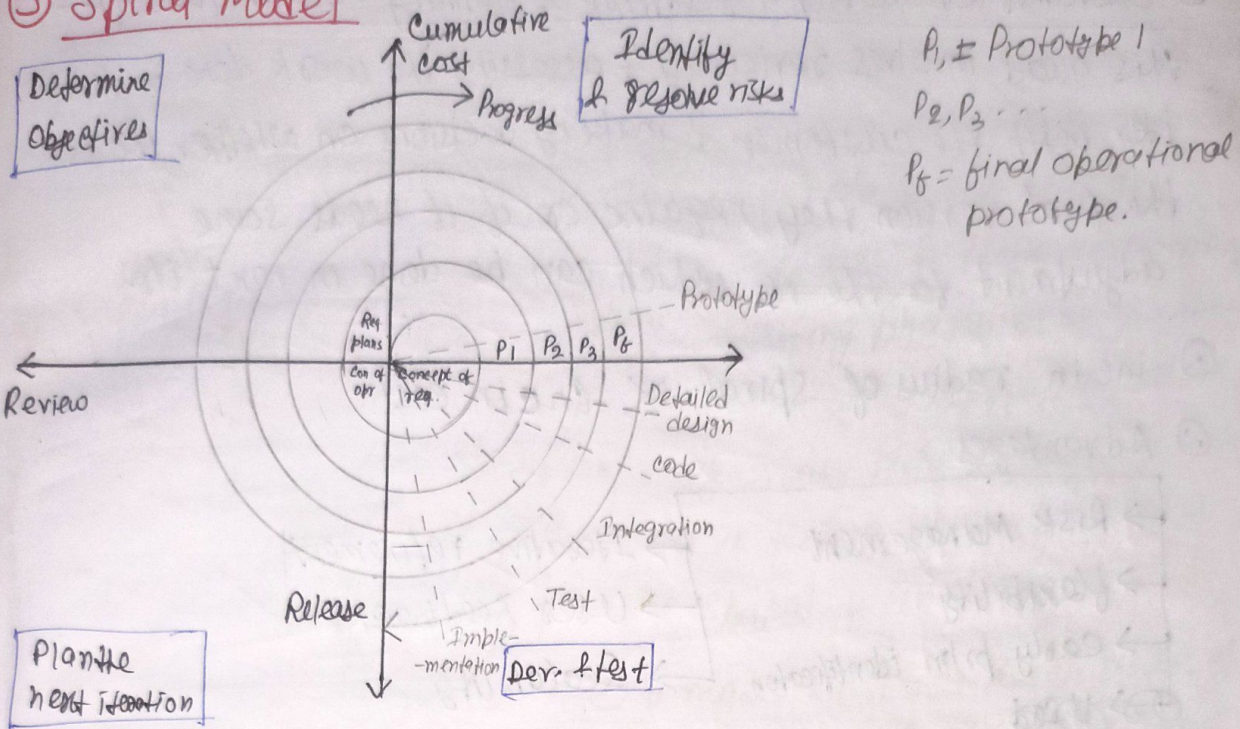
⑦ Disadvantages

- | | |
|--------------------------|-----------------------------|
| → Complex prj management | → T _{dev} is large |
| → med cost | |

⑧ Types of prj

- | | |
|-----------------------------------|-----------------------|
| → Prjs with Evolving req. | → Large/Complex prjms |
| → High-risk prjs | → User centered prjs |
| → Innovation & experimental prjs. | |

⑤ Spiral Model



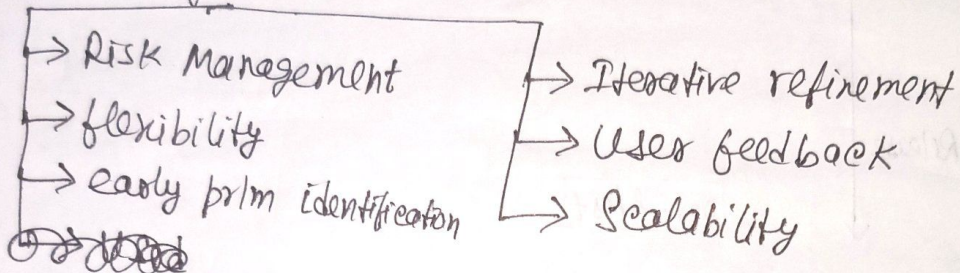
- ① It combines elements of both iterative development & the Waterfall model and also emphasizes on risk management & iterative refinement of a proj through multiple spirals.
- ② Each spiral represents a phase of development of a prototype including planning/Determine objectives (Quad 1), risk analysis & management (Quad 2), Development & testing (Quad 3) and Customer evaluation & Feedback (Quad 4).
- ③ Objective Setting/Planning (Quad 1) – this quad focuses on determining the objectives, goals, resource allocation, limitations regarding the proj, identifying alternatives. This results in clear objectives, possible approaches & resource planning.
- ④ Identifying & managing risks (Quad 2) – this quad focuses on identifying & analyzing potential risks that could impact the proj. It also aims for ^{determining} risk mitigation strategies.
- ⑤ Development & Testing (Quad 3) – this quad is the actual dev & testing phase, where after risk analysis & mitigation strategies developers put all the designing & coding efforts and then ~~it~~ integrate, test & implement the ^{new} ~~final~~ version.

① Customer evaluation & feedback & planning next itr (quad 4) -

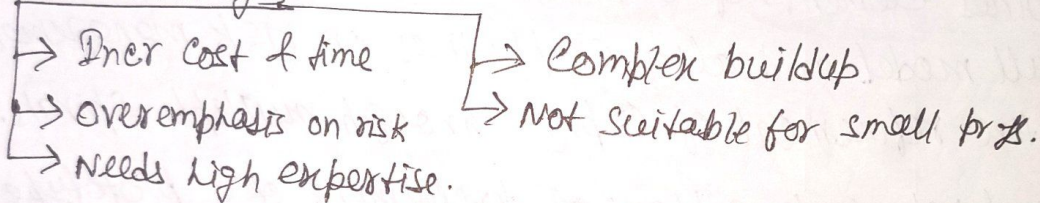
this quad involves reviewing & assessing the work done in this iter with the customer & making decisions on whether this is the final version they require or if it needs some adjustment ~~for the~~ which can be done in next itr.

② inc in radius of spiral $\propto$ inc in cost.

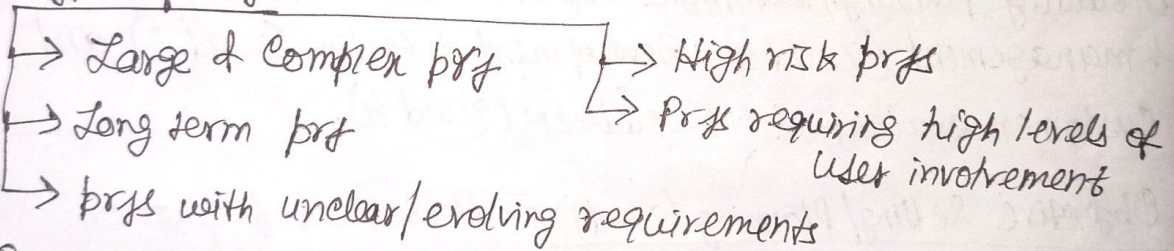
③ Advantages



④ Disadvantages



⑤ Types of Prj



⑥ Spiral Model as a Meta Model - SM is considered as a meta model because it incorporates elements from various other traditional s/w dev. models such as waterfall, iterative prototyping, evolutionary. A meta model is essentially a model that provides a framework or structure for other models.

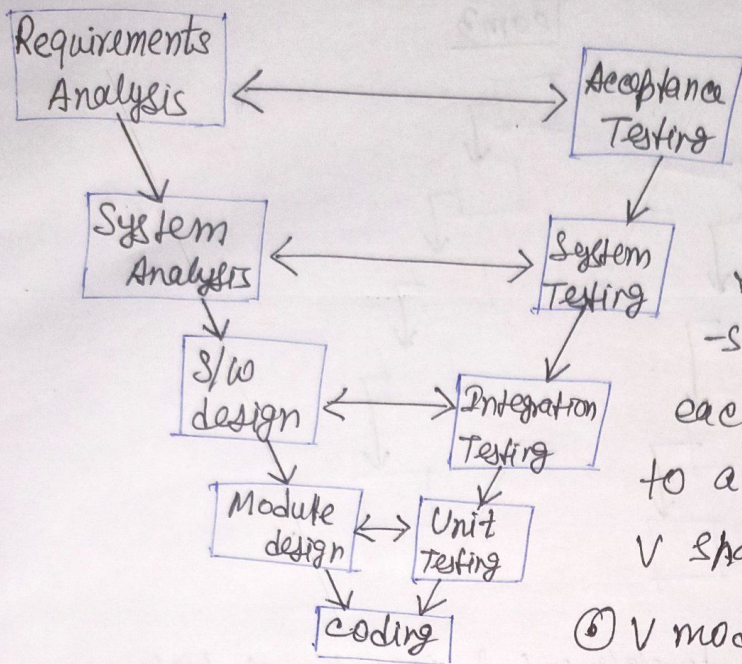
⑦ Structured ^{sequential} approach like waterfall model

⑧ iterative nature like Iterative W.M.

⑨ Focus on prototypes like PM

⑩ Uses a evolutionary approach through iteration like EM

⑥ V-Model (Verification & Validation)



⑥ It is a s/w dev process that emphasizes a parallel relationship btw dev phases & their corresponding testing phases. It is an extension of Waterfall M, where each dev phase is directly linked to a testing phase forming a V shaped when diagrammed.

⑥ V model ensures each phase of dev. is verified & validated before moving to the next phase.

⑥ Sequential process similar to WEM.

⑥ Rigid model - does not easily accommodate changes once dev. process started

⑥ Advantage

- Easy to Understand & use
- High quality product
- Early defect detection
- Clear, well defined stages with disciplined approach

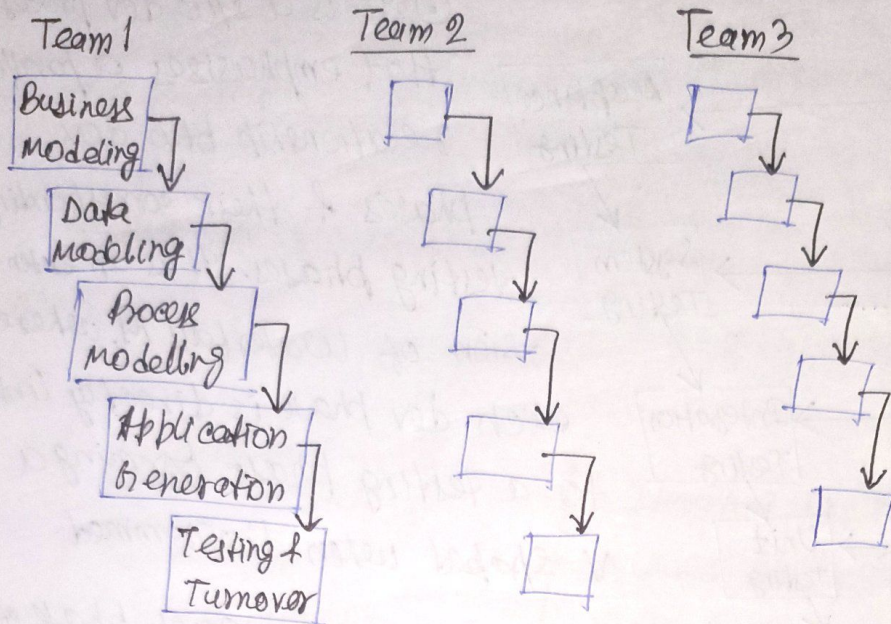
⑥ Disadvantages

- Inflexible
- Risk of unclear requirement
- Not suitable for complex & long term proj.

⑥ Types of Proj

- Well-Defined Requirements
- Proj with clear deliverable
- Small-med sized proj.

⑦ RAD (Rapid Application Development) Model



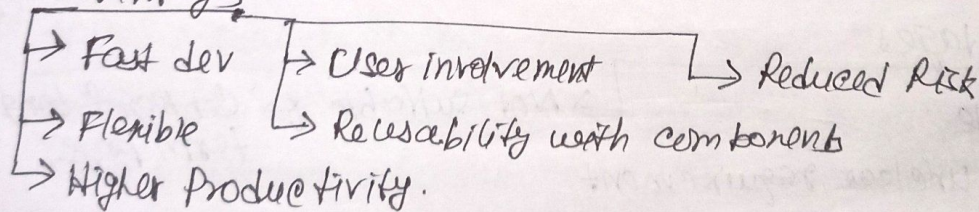
① It emphasizes on quick development & iteration of prototypes, rather than extensive upfront planning.

② It is a high speed adaptation of the linear sequential model in which rapid dev. is achieved by using component based construction along with reusability & parallel dev of components.

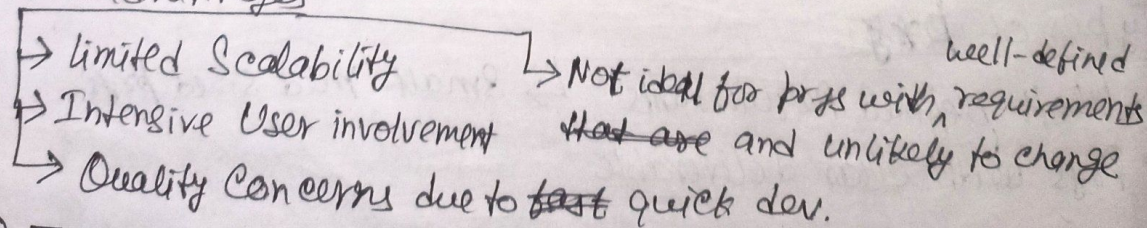
③ User ~~develop~~ involvement throughout the dev.

④ Highly flexible & adaptive.

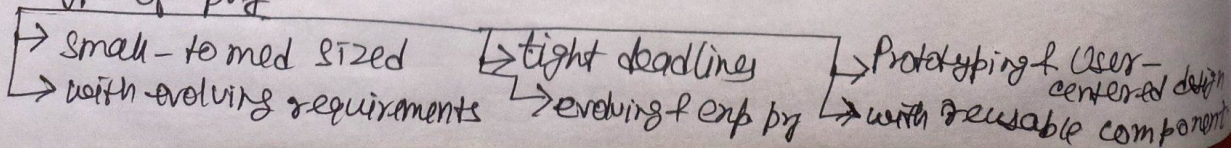
⑤ Advantages



⑥ Disadvantages



⑦ Types of prot.



MODULE 1: INTRODUCTION

Contents

1. Drawbacks of Classical Waterfall Model	1
2. Requirements Analysis and Specification	1
3. Metrics for Software Project Size Estimation.....	3
4. Function Point Metrics	4
5. COCOMO Model (Constructive Cost Model).....	6
6. COCOMO II (Constructive Cost Model II)	10
7. Cost-Benefit Analysis (CBA)	11
8. Characteristics of a Good SRS Document.....	12
9. Advantages & disadvantages of the SDLC models	12

Drawbacks of Classical Waterfall Model

1. **Inflexibility and Rigidity:** Once a phase is completed, it's difficult to go back if changes are required, making the model rigid. Each phase must be fully completed before moving to the next, leaving no room for overlap or iterative feedback.
2. **Late Discovery of Issues:** Testing begins only after development is complete, leading to late identification of issues, which can be costly to fix. Early risk mitigation is not possible since problems are only discovered in the testing phase.
3. **Poor Accommodation of Changes:** The model struggles with evolving requirements, as it assumes all requirements are known upfront. Any changes later in the process result in high costs, often requiring redesign, recoding, and retesting.
4. **High Risk of Project Failure:** Feedback is delayed until late stages, increasing the risk that the final product may not meet user expectations. Customer feedback arrives late, which can result in misaligned deliverables.
5. **Inefficient for Complex Projects:** Waterfall's linear structure is unsuitable for complex or large projects that require flexibility. It doesn't support prototypes or mock-ups to validate early assumptions.
6. **Poor Visibility and Monitoring:** Progress is hard to measure until the coding phase, making it difficult for management to gauge project status. There are no interim deliverables to provide checkpoints.
7. **Unsuitable for Long-Term Maintenance:** Maintaining and updating the system is challenging because the model doesn't support iterative refinements, and outdated documentation makes future maintenance difficult.

Requirements Analysis and Specification

Requirements analysis and specification are critical phases in the **Software Development Life Cycle (SDLC)** that aim to thoroughly understand, document, and communicate the functionality and constraints of a system. These phases ensure that developers, stakeholders, and end-users have a clear, shared understanding of what the system should achieve before design and development begin.

A. Requirements Analysis: Requirements analysis is the process of identifying, gathering, and defining the **needs and expectations** of the stakeholders for a new or modified system. The goal is to understand what the system should do (functional requirements) and the constraints or conditions under which it must operate (non-functional requirements). Key Steps in Requirements Analysis:

1. **Stakeholder Identification:** Identify the people who have a direct or indirect interest in the project (e.g., users, customers, management, regulatory authorities).
2. **Requirements Elicitation:** Gather requirements through interviews, surveys, workshops, observation, and analysis of existing systems.
3. **Categorizing Requirements:**
 - a. **Functional Requirements:** Define specific behaviours or functions the system must perform (e.g., "The system must allow users to log in using a username and password").
 - b. **Non-functional Requirements:** Define constraints on the system, such as performance, security, usability, reliability, or scalability (e.g., "The system must handle 10,000 concurrent users").
4. **Conflict Resolution:** Identifying and resolving conflicts between different stakeholders' needs or requirements.
5. **Feasibility Study:** Analysing the technical and business feasibility of the requirements to ensure that they can be implemented within constraints like time, budget, and resources.

B. Requirements Specification: The **requirements specification** is the process of documenting the results of the analysis in a clear, structured format. It formalizes the requirements so that they can be clearly communicated to all stakeholders, including designers, developers, and testers. The final document is often called a Software Requirements Specification (SRS). Key Components of Requirements Specification:

Introduction:

- a. Project overview and objectives.
 - b. Stakeholders involved in the project.
 - c. The purpose of the system.
1. **Functional Requirements:** Detailed description of each function the system must perform. Example: "The system shall allow users to reset their passwords using their registered email address."
 2. **Non-functional Requirements:** Performance, security, usability, and other constraints. Example: "The system shall encrypt user data using AES-256 encryption."
 3. **Use Cases or User Stories:** Use cases are descriptions of how users will interact with the system, often represented in a step-by-step format. User stories are short, simple descriptions of a feature from the perspective of the user. Example: "As a user, I want to be able to view my order history to track my purchases."
 4. **Assumptions and Dependencies:** Any assumptions made during the analysis and any external dependencies or integrations (e.g., with other systems or services).
 5. **Acceptance Criteria:** Clear conditions that must be met for the system to be accepted by stakeholders. These criteria help testers to validate the requirements during the testing phase. Example: "A user must be able to complete the purchase process within 2 minutes."

C. Importance of Requirements Analysis and Specification

1. **Prevents Miscommunication:** Ensures that all stakeholders, including developers, testers, and clients, have a shared understanding of what the system should do.
2. **Reduces Cost and Time:** Errors or ambiguities in requirements, if discovered later in development, can be costly to fix. Proper analysis and specification help catch potential issues early.
3. **Serves as a Reference:** The requirements specification serves as a reference throughout the development cycle, helping designers, developers, and testers to ensure that the system aligns with stakeholder expectations.
4. **Improves Project Management:** Clear requirements help project managers allocate resources, estimate timelines, and manage risks more effectively.
5. **Facilitates Testing and Validation:** Well-documented requirements are essential for creating test cases to validate that the system functions correctly. It ensures the software meets the agreed-upon functionality and performance standards.

D. Common Challenges in Requirements Analysis and Specification

1. **Changing Requirements:** Stakeholders may change their expectations as the project progresses. Managing evolving requirements is crucial, especially in long projects.
2. **Ambiguity and Vagueness:** Poorly defined requirements can lead to different interpretations by developers and testers, resulting in a system that does not meet user needs.
3. **Stakeholder Conflicts:** Different stakeholders may have conflicting requirements, which need to be resolved during analysis.
4. **Unclear Non-functional Requirements:** Sometimes, non-functional requirements like performance, security, or scalability are not clearly defined, which can lead to poor system performance or user dissatisfaction.

Metrics for Software Project Size Estimation

Estimating the size of a software project is a critical step in project planning, as it affects effort, cost, duration, and resource allocation. Various metrics are used to estimate the size of a software project. The most common size estimation metrics include:

- 1) LOC
- 2) FP Metrics

Lines of Code (LOC): The total number of lines in the source code, including executable statements and comments.

- 1) **Usage:** A basic and traditional metric, used to measure the size of the software by counting the lines of code.
- 2) **Advantages:**
 - a. Simple and easy to understand.
 - b. Provides direct correlation to effort and time needed for development.
- 3) **Disadvantages:**
 - a. Language-dependent: Different languages require different amounts of code to solve the same problem.
 - b. No focus on functionality: Doesn't capture the actual business functionality provided to the user.
 - c. Penalizes code quality: May encourage longer, less efficient code.

Function Point Metrics

Function Point (FP) is a widely used software measurement technique to estimate the size and complexity of a software project. Unlike the traditional Lines of Code (LOC) metric, which focuses on the amount of code, Function Point measures the functionality delivered to the user, providing a more user-centric and technology-independent way of estimating project effort.

A. Components of Function Points

Function Points are calculated by evaluating different aspects of a software system, categorized into five basic components:

1. **External Inputs (EI):** Inputs from users or external systems. Each input adds value by modifying data or controlling system behaviour. Example: Login Form, Registration Page, Data Upload, Search Query.
2. **External Outputs (EO):** Data or information sent to external systems or users. Includes outputs that do not update data but are generated from internal logic. Example: Report Generation, Invoice Print, Confirmation Message, Search Results Display.
3. **External Inquiries (EQ):** Interactive inputs and outputs that retrieve and display data (e.g., search results). They typically do not modify data but allow data access and presentation. Example: Search Database, Retrieve User Info, Product Availability Check, Order Status Query.
4. **Internal Logical Files (ILF):** Logical groups of data maintained and processed by the system (e.g., database tables). Files that are internal to the application and contain persistent data. Example: Customer Database, Product Catalogue, Order Records, Employee Data.
5. **External Interface Files (EIF):** Data managed by external applications, but accessed and used by the system (e.g., external databases). These files are used by the system but not maintained by it. Example: Third-party API, External Inventory System, Payment Gateway, Partner Data Feeds.

B. Steps to Calculate Function Points

1. **Determine the Unadjusted Function Points (UFP):**
 - a. Assign weights (low, average, or high complexity) to each of the five components (EI, EO, EQ, ILF, EIF).
 - b. Each component is given a weight based on its complexity, such as:
 - i. External Inputs (EI): 3 (low), 4 (medium), 6 (high)
 - ii. External Outputs (EO): 4 (low), 5 (medium), 7 (high)
 - iii. External Inquiries (EQ): 3 (low), 4 (medium), 6 (high)
 - iv. Internal Logical Files (ILF): 7 (low), 10 (medium), 15 (high)
 - v. External Interface Files (EIF): 5 (low), 7 (medium), 10 (high)
 - c. Multiply the number of components in each category by their assigned weight, then sum them up to calculate the **Unadjusted Function Points (UFP)**.
2. **Adjust for Complexity:**
 - a. After calculating the UFP, adjust the count based on 14 general system characteristics (e.g., data communications, performance, transaction rates, security).
 - b. These characteristics are rated on a scale of 0 to 5, where 0 means no influence and 5 means strong influence.
- c. Compute the **Value Adjustment Factor (VAF)/Product Complexity adjustment (PC)**:

$$\text{VAF} = 0.65 + (0.01 \times \text{Sum of Influence Factors})$$

3. Calculate Adjusted Function Points:

$$FP = UFP \times VAF$$

C. Formula Summary

1. Unadjusted Function Points (UFP):

$$UFP = \left(\sum (\text{weight for each component} \times \text{count of each component}) \right)$$

2. Value Adjustment Factor (VAF):

$$VAF = 0.65 + (0.01 \times \text{Sum of Influence Factors})$$

3. Adjusted Function Points (FP):

$$FP = UFP \times VAF$$

D. Advantages of Function Points

- **Technology Independent:** Focuses on functionality rather than programming language or technology used.
- **User-Centric:** Measures the value delivered to the user rather than focusing purely on the technical aspects.
- **Useful in Early Estimation:** Can be applied early in the development process, when LOC estimates may not be available.

E. Disadvantages of Function Points

- **Subjectivity in Complexity Rating:** Assigning complexity levels and influence factors can be subjective.
- **Requires Expertise:** Estimating function points accurately requires a good understanding of the software's functionality.

Function Point Worksheet

Measurement parameter	Count		Weighting Factor			Choice		
			simple	average	complex			
# of user inputs	11	X	3	4	6	4	=	44
# of user outputs	13	X	4	5	7	5	=	65
# of user inquiries	3	X	3	4	6	4	=	12
# of files	1	X	7	10	15	10	=	10
# of external interfaces	0	X	5	7	10	7	=	0
Count-total (UFP)=								131

Rate each factor on a scale of 0 to 5:

0 - No Influence	1 - Incidental	2 - Moderate
3 - Average	4 - Significant	5 - Essential

1. Does the system require reliable backup and recovery?	5
2. Are data communications required?	4
3. Are there distributed processing functions?	1
4. Is performance critical?	3
5. Will the system run in an existing, heavily utilized operational environment?	4
6. Does the system require on-line data entry?	1
7. Does the on-line data entry require the input transaction to be built over multiple screens or operations?	2
8. Are the master files updated on-line?	3
9. Are the inputs, outputs, files, or inquiries complex?	2
10. Is the internal processing complex?	4
11. Is the code designed to be reusable?	1
12. Are conversion and installation included in the design?	3
13. Is the system designed for multiple installations in different organizations?	2
14. Is the application designed to facilitate change and ease of use by the user?	3
Total Complexity Adjustment Value =	38

Product Complexity Adjustment (PC) = $[.65 + .01 * CAV] = 1.03$

Total Adjusted Function Point (FP) = $UFP * PC = 134.93$

Language Factor (LF) = 60

Source Lines of Code (SLOC) = $FP * LF = 8095.8$

BASIC COCOMO MODEL (ORGANIC)

LOC	8095.8		a	2.4
KLOC	8.0958	Cost per month (in Rs)	b	1.05
		50000	c	0.38

EFFORT (in PM) 21.5716816

T_{dev} (in M) 8.03185629

STAFF SIZE (in P) 3

Cost to develop (in Rs) 401592.814

COCOMO Model (Constructive Cost Model)

The **COCOMO Model** is a widely-used algorithmic software cost estimation model developed by **Barry Boehm** in 1981. It is used to estimate the effort, cost, and schedule required to develop software projects based on their size (in lines of code or function points) and various other project characteristics. Key Components of the COCOMO Model:

1. **Effort (Person-Months):** Amount of effort required to complete the project.
2. **Development Time:** The total time needed to develop the project.
3. **Software Size:** The size of the software, typically measured in **Kilo Lines of Code (KLOC)**.
4. **Effort Multipliers and Cost Drivers:** Factors that influence the project cost and effort, like team experience, tools used, etc.

A. Types of Software Products

1. **Organic:** Simple, small-sized projects with well-understood domains, limited innovation, and experienced teams. Stable requirements, low complexity, small team size, familiar development environment. **Examples:** Accounting software, inventory management systems.

2. **Semidetached:** Medium-sized projects with a mix of well-known and unfamiliar technologies and domains. Medium complexity, moderate experience in the domain, larger team size than organic projects. **Examples:** Applications with both custom and standard modules (e.g., payroll systems with additional features).
3. **Embedded:** Complex, real-time, or safety-critical software that requires close hardware-software interaction. High complexity, stringent performance, reliability, and security requirements **Examples:** Operating systems, real-time systems, defense or aerospace systems.

Product Type	Project Size	Complexity	Team Size	Familiarity	Deadline of project	Example
Organic	Small (2-50 KLOC)	Low	Small	Well-known domains	Not tight	Accounting software
Semi detached	Medium (50-300 KLOC)	Medium	Medium	Mix of familiar or unfamiliar	medium	Payroll system
Embedded	Large (300+ KLOC)	High	Large	Unfamiliar, custom	tight	Real-time operating systems

B. COCOMO Model Levels

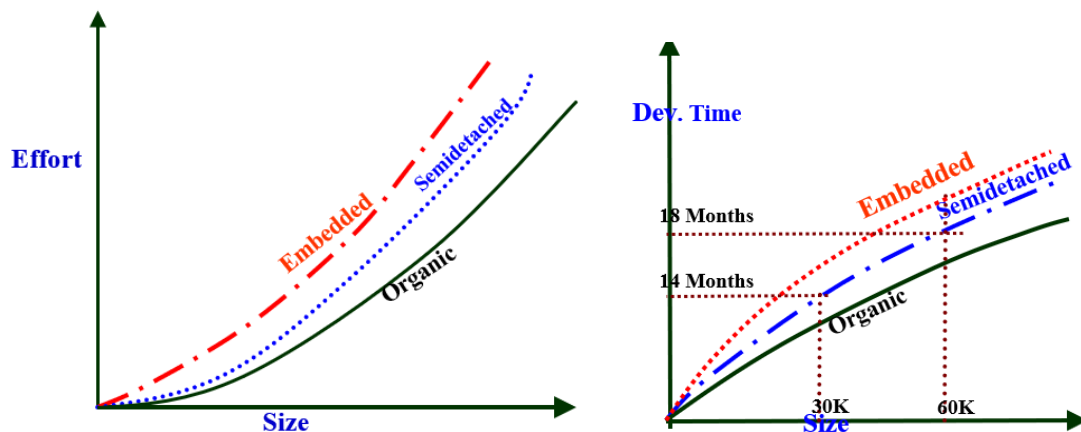
1. **Basic COCOMO:** The simplest form of the COCOMO model, where effort and cost estimation are based solely on the size of the software (in KLOC).

- **Formula:** Where **a** and **b** are constants that vary by software type (organic, semidetached, embedded).

$$Effort = a \times (KLOC)^b \quad \text{Time (TDEV)} = 2.5 \times Effort^c$$

- **Usage:** Good for early, rough estimates when limited information is available about the project.
- **Shortcomings:** Does not account for different factors like team experience, development environment, or complexity. It offers only a basic estimation.

Software project	a	b	c
Organic	2.4	1.05	0.38
Semi – detached	3.0	1.12	0.35
Embedded	3.6	1.2	0.32



For example, a 60 KLOC program can be developed in approximately 18 months, regardless of whether it is organic, semi-detached, or embedded. There is more scope for parallel activities for system and application programs, than utility programs.

- Effort = $2.4 \times (60)^{1.05}$ PM. = 176.713 PM. ➤ $T_{dev} = 2.5 \times (176.713)^{0.38}$ Months. = 17.8 Months.
- Effort = $3.0 \times (60)^{1.12}$ PM. = 294.205 PM. ➤ $T_{dev} = 2.5 \times (294.205)^{0.35}$ Months. = 18.27 Months.
- Effort = $3.6 \times (60)^{1.20}$ PM. = 489.873 PM. ➤ $T_{dev} = 2.5 \times (489.873)^{0.32}$ Months. = 18.14 Months.

2. **Intermediate COCOMO:** Builds on Basic COCOMO by including **cost drivers** or factors that influence development effort, such as product attributes, hardware constraints, personnel capabilities, and project characteristics. Provides a more detailed and accurate estimate by factoring in 15 cost drivers.

- **Formula:** Where **EAF** (Effort Adjustment Factor) is derived from a set of cost drivers (like reliability, database size, and team expertise).

$$Effort = a \times (KLOC)^b \times EAF \quad \text{Time (TDEV)} = 2.5 \times Effort^c$$

Software project	a	b	c
Organic	3.2	1.05	0.38
Semi – detached	3.0	1.12	0.35
Embedded	2.8	1.2	0.32

- The size of a software product has been estimated to be 200 KLOC. Cost drivers are as follows:

- Low Reliability 0.88
- High Product Complexity 1.15
- Low Application Experience 1.13
- High Programming Language Experience 0.95

- $EAF = 0.88 * 1.15 * 1.13 * 0.95 = 1.086$

- Semi-Detached:

- $Effort|_{corrected} = Effort * EAF = 3.0 * (200)^{1.12} * 1.086 = 1230.56 = 1231$ PM

- $Tdev|_{corrected} = 2.5 * (Effort|_{corrected})^c = 2.5 * (1231)^{0.35} = 30.16$ months = 30 months

3. **Detailed/Complete COCOMO:** The most comprehensive model, where the software project is broken down into components, and separate COCOMO estimates are applied to each module. The detailed model includes phase-sensitive effort multipliers for each stage of the project. **Usage:** Offers the most precise and refined cost estimation for large-scale projects by considering various phases (design, coding, testing).

Aspect	Basic COCOMO	Intermediate COCOMO	Complete/Advanced COCOMO
Definition	The simplest version of the COCOMO model, estimating effort based on the size of the software (KLOC).	Extends Basic COCOMO by incorporating cost drivers (effort multipliers) related to the project and development environment.	Breaks down the software project into components and applies separate cost drivers for each phase of development.

Main Inputs	KLOC (Lines of Code)	KLOC and 15 cost drivers (e.g., complexity, personnel, tools)	KLOC, cost drivers, and detailed phase breakdown of the project.
Cost Drivers	None	15 cost drivers such as product reliability, experience, team cohesion, and tools.	Phase-specific cost drivers for each stage of the development process.
Accuracy	Low to Moderate	Moderate to High, depending on the accuracy of cost driver ratings.	Highest accuracy due to detailed phase breakdown and tailored cost drivers.
Use Case	Suitable for small, simple projects with well-defined requirements where rough early estimates are needed.	Appropriate for mid-sized projects with more variability in complexity, team expertise, and development conditions.	Best for large-scale, complex projects requiring precise and detailed estimations at each phase.
Complexity	Low – simple and easy to apply.	Moderate – requires more inputs and understanding of cost drivers.	High – requires detailed project breakdown and precise phase estimation.
Flexibility	Rigid – limited to size (KLOC) and cannot accommodate project-specific variations.	More flexible than Basic, accounts for different aspects of project complexity, team skills, and environment.	Very flexible – can adapt to various project phases, complexities, and cost drivers for better precision.
Advantages	Quick and easy to use for early estimation in small projects.	More accurate due to inclusion of cost drivers, adaptable for different project types.	Provides the most accurate estimates, especially for complex, phase-sensitive projects.
Disadvantages	Doesn't consider team experience, project risks, or other influencing factors.	Relies on subjective rating of cost drivers, and is harder to apply than Basic COCOMO.	Highly complex, requires detailed inputs, and is time-consuming to implement.
Effort Drivers	Only considers KLOC (lines of code).	Considers multiple factors affecting project effort like product, personnel, and environmental factors.	Accounts for all factors considered in Intermediate COCOMO plus phase-specific attributes.
Best For	Small projects with well-understood domains.	Medium-sized projects with a mix of known and unknown factors.	Large-scale projects, or projects with complex requirements and significant uncertainty.

C. Shortcomings of Basic and Intermediate COCOMO

1. Basic COCOMO:

- **Lack of Flexibility:** Only considers software size (KLOC) without accounting for project complexity, team expertise, or other factors.
- **Inaccurate for Large Projects:** For complex or larger projects, the estimation can be inaccurate as it doesn't consider important cost drivers.

2. **Intermediate COCOMO:**

- **Cost Driver Subjectivity:** The model introduces cost drivers, but evaluating them accurately can be subjective, leading to variability in estimates.
- **Static Estimates:** Though more refined than Basic COCOMO, Intermediate COCOMO still doesn't handle dynamic changes during the project well (e.g., requirement changes).

D. Advantages:

1. **Simplicity:** Easy to understand and apply, especially in the Basic COCOMO model.
2. **Well-Defined:** Provides a structured framework to estimate cost and effort, making it useful for early project planning.
3. **Adaptability:** The Intermediate and Complete COCOMO models account for various cost drivers, making estimates more adaptable to different project types.
4. **Historical Data Usage:** COCOMO leverages historical data to improve the accuracy of cost estimation.

E. Disadvantages:

1. **Dependency on Historical Data:** Requires reliable historical data, which may not always be available or relevant for new types of projects.
2. **Limited Flexibility in Basic COCOMO:** Basic COCOMO doesn't account for cost drivers like team experience, hardware limitations, or complexity, making it unsuitable for complex projects.
3. **Complexity in Detailed COCOMO:** While offering more accurate estimates, Complete COCOMO can be very complex to apply, requiring significant effort and input data.
4. **Assumes KLOC-Based Estimation:** The model's reliance on KLOC for size measurement is outdated for modern software development practices that focus on reusable code, agile processes, and more abstract designs like microservices.

COCOMO II (Constructive Cost Model II)

COCOMO II is an advanced version of the original **COCOMO (Constructive Cost Model)**, designed to estimate the cost, effort, and schedule for software development projects. It adapts to the modern software development environment, accounting for different project types and lifecycle models. COCOMO II is more flexible and better suited for contemporary software projects that may involve prototyping, reuse, and rapid application development (RAD).

COCOMO II has three sub-models:

1. Application Composition Model:

- Used for early design stages of a project where prototypes or RAD techniques are applied.
- Focuses on projects developed using reusable components.

2. Early Design Model:

- Used when a rough design of the software system is available.
- Provides a quick, high-level estimate for effort and schedule based on early project data.

3. Post-Architecture Model:

- Applied when the project design is stable, and the full architectural details are available.
- Provides the most detailed estimates for effort, cost, and time, considering factors like scalability, complexity, and developer experience.

COCOMO II Estimation Factors

- **Scale factors:** Reflect the software development process and its scalability (e.g., team cohesion, process maturity).
- **Effort multipliers:** These adjust the effort based on specific project attributes such as product reliability, platform constraints, and personnel capability.

Cost-Benefit Analysis (CBA)

Cost-Benefit Analysis (CBA) is a financial assessment tool used to evaluate the economic feasibility of a project by comparing its costs with the anticipated benefits. **Key Steps in Cost-Benefit Analysis:**

1. **Identify Costs:**
 - **Direct Costs:** Hardware, software, developer salaries, etc.
 - **Indirect Costs:** Maintenance, training, support, etc.
 - **Opportunity Costs:** Benefits foregone by not choosing alternative projects.
2. **Identify Benefits:**
 - **Tangible Benefits:** Improved productivity, reduced errors, increased sales, etc.
 - **Intangible Benefits:** Enhanced customer satisfaction, better decision-making, competitive advantage.
3. **Quantify Costs and Benefits:** Assign monetary values to both costs and benefits. Some benefits may be hard to quantify but can still be expressed as estimates.
4. **Time Horizon:** Consider the time over which the costs and benefits will accrue (e.g., 3, 5, or 10 years).
5. **Discount Rate:** Apply a discount rate to future costs and benefits to account for the time value of money. This converts future benefits into present value terms.
6. **Perform Net Benefit Calculation:** $\text{Net Benefit} = \text{Total Benefits} - \text{Total Costs}$. If the net benefit is positive, the project is considered economically viable.
7. **Sensitivity Analysis:** Assess how sensitive the outcomes are to changes in key variables, such as cost estimates, benefit projections, or discount rates.

Advantages of CBA:

- **Simple and Intuitive:** Provides a straightforward method for evaluating the financial viability of projects.
- **Data-Driven:** Helps in making informed decisions based on quantitative financial data.
- **Broad Applicability:** Can be used across various types of projects, from software development to infrastructure investments.

Disadvantages of CBA:

- **Difficult to Quantify Intangible Benefits:** Some benefits, like user satisfaction or brand loyalty, are hard to express in monetary terms.
- **Uncertainty in Estimates:** Costs and benefits are often based on assumptions, which can lead to inaccuracies.
- **Short-Term Focus:** CBA may favour projects with short-term returns over those with long-term strategic importance.

Characteristics of a Good SRS Document

A **Software Requirements Specification (SRS)** document outlines the functional and non-functional requirements of a software system. It serves as a communication tool between stakeholders, developers, and testers. **Key Characteristics of a Good SRS Document**

1. **Correctness:** The document accurately reflects the system's requirements, capturing everything the software needs to accomplish.
2. **Completeness:** All functionalities, constraints, and specifications are clearly and fully defined. There should be no missing information that could lead to ambiguity during development.
3. **Unambiguity:** Each requirement must be clearly and precisely described to avoid misinterpretation. Terms should be well-defined, and technical jargon should be used consistently.
4. **Consistency:** The document should not have conflicting requirements or descriptions. All sections must be aligned and logically coherent.
5. **Verifiability:** Each requirement should be testable and measurable, ensuring that it can be verified through testing or inspection once the software is developed.
6. **Traceability:** The origin of each requirement should be clear, and the document should provide links between each requirement and its corresponding system component or business need.
7. **Modifiability:** The document should be structured in a way that allows easy updates as requirements evolve. Clear organization and formatting contribute to this.
8. **Feasibility:** The requirements described must be realistic and achievable within the constraints of technology, time, and budget.
9. **Prioritization:** Requirements should be ranked by their importance to the project's success, helping guide developers in delivering the most critical functionality first.

Advantages & disadvantages of the SDLC models

SDLC Model	Advantages	Disadvantages
Classical Waterfall Model	1. Simple and easy to understand 2. Well-structured stages 3. Clear documentation with well-defined input and output. 4. Well suited for low and fixed price contracts.	1. Rigid, no revisions 2. No changes once the stage is completed 3. High development time. 4. Not ideal for the complex and uncertain projects 5. High risk and uncertainty.

Iterative Waterfall Model	1. Allows feedback at each stage 2. Early detection of issues 3. Structured and disciplined approach. 4. Better risk management. 5. More reliable and higher quality of final product.	1. More complex than classical 2. Limited no of iterations 3. Large development time. 4. Risk of over iteration. 5. Managing iterations and feedback is complex.
Prototyping Model	1. Early detection of issues 2. Provides working prototype early 3. Better requirement understanding. 4. Risk reduced.	1. Prototype may lack structure 2. Increases cost and time 3. Highly complex. 4. Due to quick builds prototype lack perfection structure and scalability.
Evolutionary Model	1. Evolving requirements 2. Higher flexibility. 3. Improved user involvement. 4. Improved risk. 5. Useful for larger size projects	1. Difficult to manage constant changes 2. Time of development is large.
Spiral Model	3. Focuses on risk management 4. Combines iterative and feedback development 5. Early problem identification.	1. Complex and costly 2. Unsuitable for small projects 3. Over emphasis of risk. 4. May suffer due to need of higher expertise.
RAD Model	1. Quick delivery 2. Encourages user feedback 3. Reduce the risk.	1. Needs skilled developers 2. Not ideal for large projects 3. Quality concerns due to quick development.
V Model	1. Emphasizes verification and validation 2. Suitable for small projects 3. Easy to understand and use.	1. Rigid, handles changes poorly 2. Testing starts after development 3. Risk of unclear requirements. 4. Not suitable for complex and long-term projects.
Agile Model	1. Highly flexible 2. Continuous feedback and collaboration	1. Requires strong team collaboration 2. Less predictable outcomes

MODULE 2: INTRODUCTION

Contents

1. Data Flow Diagram (DFD)	1
2. Logical vs Physical DFD	1
3. Control Flow Diagram / Flowchart	2
4. Decision Tree	3
5. Decision Table.....	3
6. Software Project Plan	3
7. Characteristics of a Good SPMP Document	4
8. Function-Oriented vs Object-Oriented Design.....	5
9. Problem Partitioning.....	6

Data Flow Diagram (DFD)

A Data Flow Diagram (DFD) is a graphical representation of data flow within a system, illustrating how input data is processed to generate output data. It helps in understanding how data moves through a system and interacts with different components. **Components of a DFD**

- **External Entities** (Source/Sink): Represent external systems, organizations, or people that interact with the system (e.g., users, external databases).
- **Processes**: Represent activities or functions where data is processed (e.g., calculations, database queries).
- **Data Stores**: Represent storage locations (e.g., databases, files) where data is kept for future use.
- **Data Flows**: Represent the movement of data between processes, data stores, and external entities.
- **DFD Levels**: DFDs are divided into levels to represent the system at varying levels of detail:
 - **Level 0 (Context Diagram)**: This is the highest level, providing a very high-level overview of the entire system. It shows the system as a single process and identifies the primary input and output data flows between the system and external entities.
 - **Level 1**: Breaks down the single process from the Level 0 diagram into more detailed subprocesses. It provides a deeper understanding of the major processes in the system.
 - **Level 2**: Further breaks down the Level 1 processes into more granular subprocesses, showing how each function works and how data moves within smaller units of the system.

Logical vs Physical DFD

Aspect	Logical DFD	Physical DFD
Definition	A Logical DFD focuses on the business functions and processes without considering the actual system architecture.	A Physical DFD focuses on how the system is implemented, including hardware, software, files, and people.

Purpose	To represent the business processes and data flow independent of physical implementation.	To depict how the system will be physically implemented in terms of resources and hardware.
Components	Data Processes, Data Stores, External Entities, Data Flows	Processes, Data Stores, External Entities, Data Flows, Files, Devices, Hardware, and People
Level of Abstraction	Higher level of abstraction, focusing on what the system does (the <i>logic</i> of operations).	Lower level of abstraction, focusing on how the system will be built and how it operates physically.
Focus	Describes what needs to be done in the system, emphasizing system requirements from a functional perspective.	Describes how the system will be implemented, including the physical aspects of the system's environment.
Technology Independent	Does not consider the hardware, software, or physical architecture involved in the system.	Considers actual hardware, software, and other technologies that will be used in the system implementation.
Examples of Components	- Process (e.g., "Process Orders") - Data Store (e.g., "Customer Data") - Data Flow (e.g., "Order Information") - External Entity (e.g., "Customer")	- Process (e.g., "Order Processing System") - Data Store (e.g., "Customer Database") - Physical Data Flow (e.g., "Data Transfer between Database and Application") - Physical Device (e.g., "Server")
When Used	Used in the early stages of system design to understand and model business needs without worrying about implementation.	Used in the later stages of system design to plan how the system will be constructed and the resources required for implementation.
Audience	Mostly intended for business analysts, stakeholders, and users who need to understand the business processes.	Intended for system developers, engineers, and IT staff involved in system implementation.
Example	A logical DFD might show how customer orders are processed in a retail system without specifying the database or software used.	A physical DDF would specify the database (e.g., MySQL), the server (e.g., Linux-based), and the data paths between components.

Control Flow Diagram / Flowchart

A **flowchart** or **control flow diagram** is a graphical representation used to depict the flow of control in a process or system. It visually describes the sequence of steps, decisions, and actions that must be taken to achieve an objective. **Components of a Flowchart**

- **Start/End (Oval):** Indicates the beginning and end points of the process.
- **Process (Rectangle):** Represents a step or action in the process.

- **Decision (Diamond):** Represents a decision point where a yes/no question or a condition is evaluated.
- **Input/Output (Parallelogram):** Denotes input or output data to/from the system.
- **Arrows:** Indicate the flow direction from one step to the next.

Example Flowchart

Decision Tree

A **decision tree** is a tree-like graphical representation used to make decisions based on certain conditions. It is often used in machine learning and decision-making scenarios, helping to visualize and evaluate the consequences of choices. **Components of a Decision Tree**

- **Root Node:** The starting point, representing the main decision or condition.
- **Internal Nodes:** Represent the conditions or tests applied to the data.
- **Branches:** Show the possible outcomes of the conditions.
- **Leaf Nodes:** Represent the final decision or outcome of the decision-making process.

Example of a Decision Tree

Decision Table

A **decision table** is a tabular representation that maps different conditions and the actions that result from them. It is especially useful in situations where multiple conditions can result in different actions. **Components of a Decision Table**

- **Conditions:** Represent the criteria or rules that need to be checked.
- **Actions:** Represent the corresponding actions to be taken based on the conditions.
- **Rules:** The columns that map specific combinations of conditions to their corresponding actions.

Example of a Decision Table

- If the customer is employed **and** has sufficient income (Rule 1), the loan is **approved**.
- For other combinations of employment and income (Rules 2, 3, 4), the loan is **denied**.

Conditions	Rule 1	Rule 2	Rule 3	Rule 4
Is Customer Employed?	Yes	Yes	No	No
Has Sufficient Income?	Yes	No	Yes	No
Action	Approve	Deny	Deny	Deny

Software Project Plan

A **Software Project Plan (SPP)** outlines the key steps, resources, timeline, and goals for a software development project. It provides a roadmap for how the project will be executed, managed, and delivered. A well-defined project plan ensures that the team stays on track, risks are minimized, and objectives are met within the given constraints. **Key Components of a Software Project Plan:**

1. **Project Scope:** Defines what the project will deliver, including the features, functionalities, and constraints. It outlines the project's boundaries and goals.
2. **Project Objectives:** The specific, measurable, achievable, relevant, and time-bound (SMART) goals the project is aiming to achieve.
3. **Schedule and Milestones:** A detailed timeline of project activities, including the start and end dates, along with key milestones to track progress.
4. **Resource Allocation:** Identifies the human, financial, and technical resources required for the project. This includes team members, budgets, and hardware/software needs.
5. **Risk Management Plan:** Identifies potential risks and outlines strategies for mitigating them. Includes contingency plans for managing risks that materialize.
6. **Communication Plan:** Specifies how project updates, progress reports, and stakeholder communications will be handled throughout the project lifecycle.
7. **Quality Assurance:** Defines the approach for ensuring the final software product meets quality standards, including testing strategies, reviews, and validation processes.
8. **Project Monitoring and Control:** Describes how progress will be tracked and measured, and how deviations from the plan will be addressed to keep the project on track.
9. **Change Management:** Defines the process for handling changes in project scope, requirements, or timelines, ensuring that changes are controlled and documented.

Characteristics of a Good SPMP Document

A **Software Project Management Plan (SPMP)** document outlines how a software project will be executed, monitored, and controlled. It serves as a guide for managing the project's resources, timelines, risks, and deliverables. **Key Characteristics of a Good SPMP Document**

1. **Clarity and Precision:** Clearly define project goals, scope, schedule, budget, and deliverables to avoid misunderstandings.
2. **Comprehensive Planning:** Cover resource allocation, risk management, quality assurance, and communication for all project phases.
3. **Realistic Goals:** Set achievable objectives and milestones based on available time, budget, and resources.
4. **Flexibility:** Allow for adaptability with contingencies for unexpected changes.
5. **Roles and Responsibilities:** Clearly define each team member's role to prevent confusion and ensure accountability.
6. **Milestones and Deliverables:** Specify critical milestones and timelines for deliverables.
7. **Risk Management:** Include a risk management plan with mitigation and contingency strategies.
8. **Resource Management:** Detail the allocation of human, financial, and technical resources.
9. **Performance Monitoring:** Define metrics and checkpoints to track project progress and maintain scope.
10. **Communication Plan:** Set communication guidelines for frequency, format, and participants in updates and meetings.

Function-Oriented vs Object-Oriented Design

Aspect	Function-Oriented Design Approach	Object-Oriented Design Approach
Definition	Focuses on decomposing the system into a set of functions or procedures that perform specific tasks.	Focuses on decomposing the system into a collection of interacting objects that encapsulate data and behaviour.
Main Concept	Organized around functions (procedures) and algorithms that transform inputs into outputs.	Organized around objects that represent real-world entities and have attributes (data) and methods (behaviour).
Decomposition	Breaks down the system into functions or modules, which are defined by their operations.	Breaks down the system into objects and classes, each representing a real-world entity with both data and behavior.
Modularity	Modules correspond to functions, and interaction happens through function calls.	Modules correspond to classes or objects, and interaction happens through object methods (messages).
Data Handling	Data is passed between functions, and often there is no central place to store data.	Data is encapsulated within objects, and access to it is controlled via methods (encapsulation).
Design Focus	Focuses on processes and how the data flows through the system.	Focuses on data (in the form of objects) and the interactions between these objects.
State Management	Functions typically operate on data, but they don't retain state information across calls.	Objects maintain their own state through attributes and can change their state through methods.
Reuse	Reusability is achieved by breaking down tasks into small, reusable functions.	Reusability is achieved through inheritance, polymorphism, and composition of objects.
Flexibility & Extensibility	Harder to adapt as changes in one function can require changes in other functions due to tightly coupled logic.	More flexible and extensible due to inheritance, encapsulation, and polymorphism, which allows changes in objects without affecting others.
Example	Example: A payroll system designed around functions like calculateSalary() and deductTaxes().	Example: A payroll system designed with classes like Employee, Salary, and Tax where each object interacts and manages its own data.
Design Representation	Typically represented using data flow diagrams (DFDs) and flowcharts that show how functions interact.	Typically represented using UML diagrams (Class Diagrams, Sequence Diagrams, etc.) that model objects and their interactions.
Primary Goal	To transform inputs into desired outputs using functional decomposition.	To model the problem domain by creating objects that interact with each other to fulfill system requirements.
Coupling	Tends to have higher coupling because functions depend on data being passed between them.	Tends to have lower coupling due to encapsulation, where objects manage their own data.
Abstraction	Focuses on the abstraction of processes and algorithms.	Focuses on the abstraction of real-world entities as objects.

Maintenance	Maintenance can be challenging due to tight coupling between functions and data dependencies.	Easier to maintain due to modularity, encapsulation, and clear separation of responsibilities.
Real-World Mapping	Limited real-world mapping since it revolves around functional procedures.	Direct real-world mapping by modelling entities as objects that interact with each other.

Problem Partitioning

Problem Partitioning is a design and analysis technique used in software engineering and systems development. It involves breaking down a complex problem into smaller, more manageable sub-problems or components. This approach can enhance understanding, simplify the design process, and facilitate easier implementation and maintenance.

To make complex problems more manageable, understandable, and solvable by focusing on smaller, discrete components.

Benefits

1. Simplifies problem analysis and design
2. Enhances clarity and understanding
3. Facilitates parallel development
4. Improves maintainability
5. Supports better testing and debugging

Types of Partitioning

1. **Functional Partitioning:** Dividing based on different functions or operations.
2. **Data Partitioning:** Dividing based on different data sets or structures.
3. **Task Partitioning:** Dividing based on different tasks or responsibilities.
4. **Layered Partitioning:** Dividing based on different layers of abstraction (e.g., presentation layer, business logic layer, data access layer).

Implementation of Problem Partitioning

1. **Identify the Overall Problem:** Understand the complete problem that needs to be solved.
2. **Break Down the Problem:** Divide the main problem into smaller, manageable sub-problems or modules.
3. **Analyse Sub-Problems:** Assess each sub-problem to understand its requirements, constraints, and how it interacts with other components.
4. **Define Interfaces:** Establish clear interfaces for communication between the different sub-problems or modules.
5. **Implement and Test:** Develop each partition independently, followed by integration testing to ensure they work together as expected.

MODULE 3: CODING & DOCUMENTATION

Contents

1. Structured Programming.....	1
2. Object-Oriented Programming (OOP).....	2
3. Information Hiding.....	2
4. Reuse	3
5. System Documentation.....	4
6. Software testing.....	5
7. Software Testing Life Cycle (STLC).....	6
8. Types of Testing	8
9. Functional Testing	8
10. Non-Functional Testing	10
11. Testing Strategies and Levels.....	12
12. Reliability Assessment.....	14
13. Validation & Verification (V&V)	14
14. 12 Key Metrics in Software Testing	15
15. Monitoring & Control in Software Testing.....	15

Structured Programming

Structured programming is a programming paradigm aimed at improving clarity, quality, and development time. Here are the key aspects to study:

1. **Top-Down Design:** Structured programming promotes breaking down a program into smaller, simpler sub-tasks. This approach, called "top-down design," allows developers to focus on smaller components of a program individually, making it easier to manage complex systems.
2. **Control Structures:** Structured programming relies on three primary control structures to organize code logically and sequentially:
 - **Sequence:** Executes statements one after another in a fixed order.
 - **Selection:** Uses decision-making structures like if, else, and switch statements to branch the code based on conditions.
 - **Iteration:** Repeats a block of code using loops (for, while, or do-while), allowing operations to be repeated until certain conditions are met.
3. **Modularity:** Programs are organized into modules, or functions, each responsible for a specific task. This modular approach not only makes the code easier to read and maintain but also helps in identifying and isolating bugs.
4. **Readability and Maintainability:** Structured programming emphasizes writing clear and simple code. The use of modules, along with logical control structures, improves readability and maintainability. Well-organized code is easier for others to understand, test, and update.

5. **Error Reduction:** Because structured programming encourages dividing tasks into smaller functions, it can reduce errors. Debugging is simpler when issues are isolated to smaller code blocks rather than large, complex sections.
6. **Example Use Cases:** Structured programming is especially useful in procedural programming languages like C, Pascal, and early versions of BASIC, which are designed to support structured programming principles.

Object-Oriented Programming (OOP)

Object-Oriented Programming (OOP) is a paradigm designed to model complex systems using "objects" that represent real-world entities. Key concepts of OOP include:

1. **Classes and Objects:**
 - **Classes** are blueprints or templates for creating objects. Each class defines the properties (attributes) and behaviours (methods) that an object created from it will have.
 - **Objects** are instances of classes. Each object has its own data and can perform actions defined in its class, making it a unique entity within a program.
2. **Encapsulation:** Encapsulation is the bundling of data (attributes) and methods (functions) within a class, restricting direct access to some of the object's components. This control over data access is essential for preventing unintended interactions and maintaining data integrity. It also supports "information hiding," as users of a class don't need to understand its inner workings.
3. **Inheritance:** Inheritance allows a class (subclass or derived class) to inherit properties and methods from another class (superclass or base class). This enables code reuse, as common functionality in a superclass can be shared across multiple subclasses, reducing redundancy and improving maintainability.
4. **Polymorphism:** Polymorphism allows objects of different classes to be treated as objects of a common superclass, especially when those classes share a common interface. Two key types:
 - **Compile-time polymorphism** (method overloading): Allows multiple methods in a class to have the same name but different parameters.
 - **Runtime polymorphism** (method overriding): Allows subclasses to provide a specific implementation of a method that is already defined in its superclass.
5. **Abstraction:** Abstraction involves simplifying complex systems by modelling classes based on relevant attributes and actions. This allows a developer to focus on high-level interactions without needing to know low-level details, facilitating both design and functionality.
6. **Modularity and Reusability:** OOP supports modular design through classes and inheritance. Once a class is defined, it can be reused in other programs, reducing development time. Inheritance further enhances reusability, as it allows new classes to build upon existing ones.
7. **Example Use Cases:** OOP is widely used in languages like Java, C++, Python, and C#. It's beneficial in scenarios requiring complex data models, user interfaces, or any program where objects naturally map to real-world entities.

Information Hiding

Information hiding is a core principle in software design that focuses on restricting access to internal details of a module, class, or function to protect data integrity and prevent unintended interference

from other parts of the program. Information hiding involves encapsulating the inner workings of a module, class, or function, making only necessary information visible to the outside.

1. Why Information Hiding Matters:

- **Data Integrity:** Hiding implementation details prevents unintended data access or modifications, enhancing code reliability.
- **Reduced Complexity:** Users do not need to understand internal workings; they only interact with the inputs, outputs, and purpose of a function or class.
- **Maintainability:** Internal changes do not impact other parts of the code if the public interface stays the same, making updates and refactoring easier.

2. Examples of Information Hiding:

- **Classes and Private Variables:** In OOP, private attributes are accessed through “getter” and “setter” methods, allowing controlled access.
- **Encapsulation in Modules:** Each module hides its data and functions, exposing only essential interfaces for interaction.

3. Benefits of Information Hiding:

- **Improved Security:** Reduces the chance of accidental access to sensitive data.
- **Clear Interfaces:** Users only see what they need to interact with, which makes code easier to use and understand.
- **Easier Debugging:** Since each module is self-contained, debugging and testing become easier, as problems are likely confined to a single module rather than affecting the entire system.

Reuse

Reuse in software engineering refers to the practice of using existing code, components, or systems in new projects or parts of a project, which reduces redundancy and saves time and effort. **Types of Reuse:**

- **Code Reuse:** Reusing code modules or libraries across projects, like using sorting algorithms without rewriting them.
- **Component Reuse:** Reusing components (e.g., UI elements like buttons) or complex modules in multiple projects.
- **Design Patterns:** Reusable solutions to common design problems, such as the Singleton (single instance), Factory (object creation), and Observer (event handling) patterns.

1. Advantages of Reuse:

- **Time and Cost Efficiency:** By reusing existing code or components, developers can save significant time that would otherwise be spent on coding from scratch.
- **Improved Reliability:** Reused code is often already tested and debugged, which can lead to fewer errors in the final product.
- **Standardization:** Reuse encourages a consistent structure and reduces variability, making systems easier to understand, maintain, and extend.

2. Challenges of Reuse:

- **Integration Issues:** Code reuse can be difficult if the reused code has dependencies or conflicts with the new system.
- **Compatibility and Maintenance:** Reused code may need updates to stay compatible with future versions or new project features.

3. Best Practices for Reuse:

- **Modular Design:** Writing code in small, independent modules or classes increases the potential for reuse.
- **Documentation:** Clearly documenting reusable components helps other developers understand and integrate them more easily.
- **Use of Libraries and Frameworks:** Using libraries for common functionalities like data processing or UI components encourages reuse and reduces duplication.

4. Examples of Reuse in Software Development:

- **Standard Libraries:** Standard libraries in programming languages (e.g., Java's `java.util` package or Python's `math` module) offer reusable functionalities that developers can directly implement.
- **API and SDK Reuse:** Application Programming Interfaces (APIs) and Software Development Kits (SDKs) provide pre-built functionality, which developers can incorporate into new applications, enabling faster development.

System Documentation

System documentation is the comprehensive record of a software system's design, functionality, usage, and maintenance procedures. It serves as a vital resource for developers, users, and stakeholders, facilitating understanding, maintenance, and enhancement of the software.

1. Purpose of System Documentation:

- **Communication:** Documentation communicates essential information about the system's structure, purpose, and functionality to users and developers.
- **Maintenance:** It aids developers in maintaining and updating the system over time, even if the original developers are no longer available.
- **Training:** New team members can get up to speed quickly by reviewing the documentation.
- **Troubleshooting:** Documentation provides crucial information for debugging and resolving issues.

2. Types of System Documentation:

- **Requirements Documentation:**
 - Specifies what the system is intended to do, including functional and non-functional requirements.
 - Outlines specific features, user needs, and performance expectations to guide the development process.
- **Design Documentation:**
 - Provides details on the software's architecture and design patterns.
 - Includes diagrams such as UML (Unified Modelling Language) diagrams, flowcharts, and data models that describe the structure and interaction between components.
- **Technical Documentation:**

- Includes details on coding standards, programming languages, APIs, and third-party libraries.
- Guides developers on how to build, deploy, and troubleshoot the system.
- Covers specifics like database schemas, error handling mechanisms, and security measures.
- **User Documentation:**
 - Aimed at end-users, providing instructions on using the software.
 - Includes user manuals, help guides, FAQs, and tutorials to assist users in navigating and utilizing the system's features.
- **Maintenance Documentation:**
 - Contains information necessary for maintaining and updating the system.
 - Covers topics like system configurations, hardware requirements, troubleshooting guides, and backup procedures.
 - Important for extending functionality, updating components, and migrating systems over time.

3. Best Practices in System Documentation:

- **Clarity and Consistency:** Documentation should be easy to understand, consistent in terminology, and free of technical jargon where possible.
- **Modularity:** Organize documentation in sections based on specific aspects (e.g., setup, configuration, troubleshooting) to make it accessible and easy to navigate.
- **Regular Updates:** Documentation should evolve with the system. It is essential to update it with every major change, bug fix, or feature addition.
- **Automation and Tools:** Use tools like Javadoc, Doxygen, or Sphinx for automatic documentation generation, especially for code documentation, to ensure consistency and reduce manual work.
- **Version Control:** Document changes and maintain version histories to track the evolution of the system and provide context for future modifications.

4. Documentation Standards:

- Using standardized formats and templates helps ensure that documentation is uniform, comprehensive, and easy to follow.
- Industry standards like IEEE (Institute of Electrical and Electronics Engineers) and ISO (International Organization for Standardization) provide guidelines for documentation structure, content, and quality.

5. Benefits of Comprehensive Documentation:

- **Improves Team Efficiency:** Well-documented systems allow team members to quickly understand and work on the project.
- **Reduces Errors and Bugs:** Clear guidelines on functionality and usage reduce misunderstandings that could lead to bugs.
- **Simplifies Future Enhancements:** With detailed documentation, future upgrades and new feature implementations are easier to manage.

Software testing

Software testing is a process used to ensure that software behaves as expected and meets the desired quality standards. It involves executing software to identify defects, verify that the software functions correctly, and ensure it fulfils user requirements. **Key Aspects of Software Testing:**

1. Objectives of Testing:

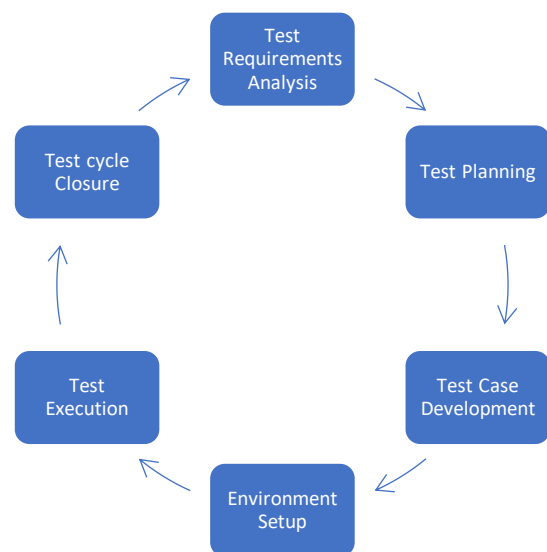
- **Find Defects:** Detect and correct bugs or issues in the software.
- **Validate Functionality:** Ensure the software works according to its specifications and requirements.
- **Ensure Quality:** Confirm that the product meets the standards of performance, security, and usability.
- **Provide Confidence:** Testing helps stakeholders have confidence that the software is working as intended.

Software Testing Life Cycle (STLC)

The **Software Testing Life Cycle (STLC)** refers to the series of phases or steps that are followed to test the software system thoroughly. It includes activities from the initial requirement analysis to the final test cycle closure. Below are the phases in the STLC:

1. Test Requirements Analysis: This is the first phase where the testing team analyses the requirements of the software to understand the scope of testing and identify the necessary resources.

- Understanding business and technical requirements.
- Identifying testable requirements.
- Reviewing documents like requirements, user stories, and acceptance criteria.
- Identifying testing objectives and deliverables.
- Defining test environments and test data.
- **Goal:** To ensure the requirements are clear, complete, and testable.



2. Test Planning: In this phase, a detailed **Test Plan** is created. The test planning phase involves deciding the testing strategy, resources, scope, and schedule.

- Defining the scope of testing (functional, non-functional, integration, etc.).
- Selecting the testing tools to be used.
- Estimating the effort and timeline for the testing.
- Assigning roles and responsibilities to the testing team.
- Identifying risks and mitigation strategies.
- Defining test deliverables (test cases, test reports, etc.).
- **Goal:** To create a comprehensive plan that guides the testing process and provides clarity on how testing will be conducted.

3. Test Case Development: Once the test plan is in place, the next phase involves **creating detailed test cases**. These test cases will be used to validate the software according to the requirements.

- Writing test cases based on the requirements and design documents.
- Preparing test data required to execute test cases.
- Reviewing test cases with stakeholders to ensure accuracy.
- Creating test scripts (for automated tests).
- **Goal:** To develop test cases and scripts that will effectively validate the software.

4. Environment Setup: This phase involves setting up the **test environment**, which includes hardware, software, network configurations, and tools required for testing.

- Preparing the required hardware and software environments.
- Setting up test databases and test data.
- Installing necessary test tools (e.g., bug tracking, automation tools).
- Ensuring the test environment mirrors the production environment as closely as possible.
- **Goal:** To ensure that the test environment is ready to execute the test cases and that all necessary tools and data are available.

5. Test Execution: During this phase, the actual **test cases are executed**, and the results are recorded. Test execution is done by running the test cases on the software, either manually or using automated test scripts.

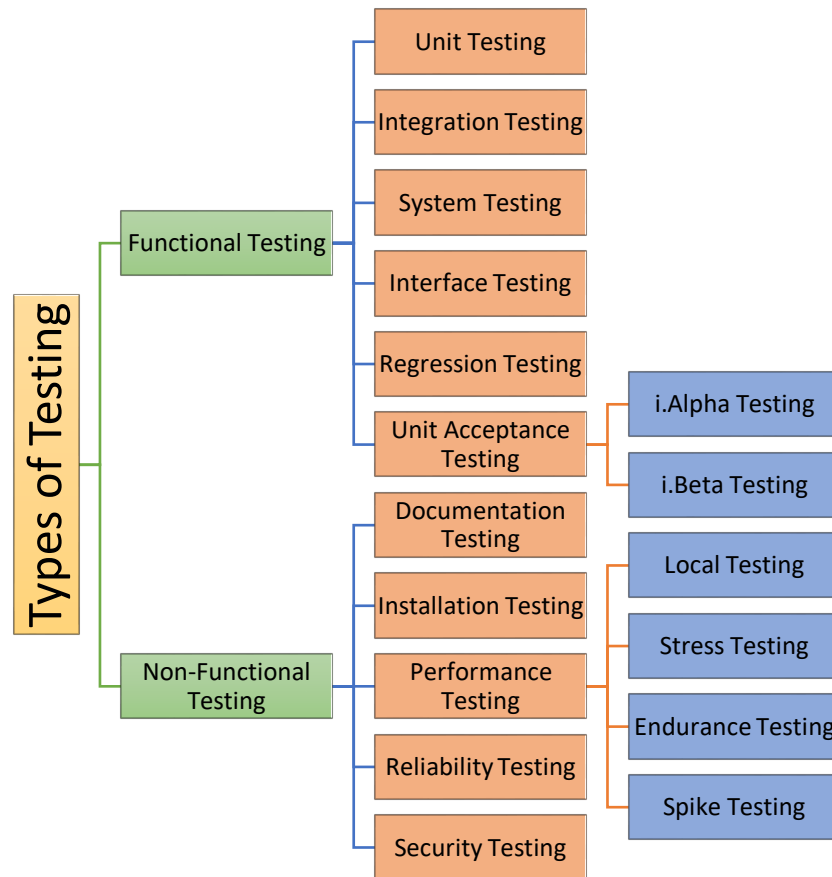
- Executing the test cases as per the test plan.
- Recording the actual results and comparing them with the expected results.
- Reporting any defects or issues discovered during testing.
- Retesting after defects are fixed.
- **Goal:** To validate whether the software works as expected and to identify any defects or issues.

6. Test Cycle Closure: Once the testing activities are complete, the **test cycle closure** phase is executed. This phase involves documenting the testing results and closing the test phase.

- Preparing test summary reports and documentation.
- Finalizing and storing test deliverables (test cases, test logs, defect reports).
- Conducting a test closure meeting with the project team.
- Reviewing the testing process to identify improvements for future projects.
- Closing open issues and ensuring that all test cases have been executed.
- **Goal:** To formally close the testing phase, ensuring that all activities are complete and that the software has been thoroughly tested.

Phase	Activities	Goal
Test Requirements Analysis	Analysing the requirements and defining test scope	Ensure requirements are testable
Test Planning	Developing test plans, schedules, and resource allocation	Plan how testing will be done
Test Case Development	Writing detailed test cases and preparing test data	Develop test scripts and data
Environment Setup	Setting up the test environment and necessary tools	Ensure environment readiness
Test Execution	Executing test cases and reporting defects	Validate functionality and identify defects
Test Cycle Closure	Finalizing reports, conducting closure meetings	Close the testing phase

Types of Testing



Functional Testing

Functional testing evaluates whether the software behaves according to the functional requirements. Types of functional testing:

- 1. Unit Testing:** Tests individual units or components of a software application to verify each part functions as intended in isolation.
 - **Scope:** Focuses on the smallest testable parts of the application, such as functions, methods, or classes.
 - **Performed By:** Typically done by developers during the development phase.
 - **Objective:** Ensures that each function or method performs as expected with different inputs, catching errors early.
 - **Tools:** Popular unit testing tools include JUnit (Java), NUnit (.NET), and PyTest (Python).
- 2. Integration Testing:** Tests interactions between integrated units or components to check if they work together as intended.
 - **Scope:** Covers combined components, such as multiple modules, APIs, or databases working in conjunction.
 - **Approach:** Common approaches include **Top-Down** (test top modules first), **Bottom-Up** (test lower modules first), and **Big Bang** (testing all components together).
 - **Objective:** Identifies interface-related defects and compatibility issues when components interact.
 - **Example:** Ensuring that data flows correctly from one module to another in a multi-layered architecture.
 - **Tools:** Tools like JUnit, TestNG, and SoapUI are used for integration testing.

3. **System Testing:** Tests the complete, integrated system to evaluate if it meets specified requirements.
 - **Scope:** Covers the entire application, including all modules, components, and interactions with external systems.
 - **Performed By:** Usually done by a dedicated testing team after integration testing is complete.
 - **Objective:** Validates the overall functionality and behaviour of the entire system under real-world conditions.
 - **Focus Areas:** Includes functionality, user interactions, performance, security, and compatibility.
 - **Tools:** System testing tools include Selenium, QTP, and TestComplete.
4. **Interface Testing:** Specifically tests the interfaces between components or systems, such as APIs, user interfaces, or third-party integrations.
 - **Scope:** Focuses on data flow and communication between systems and modules.
 - **Objective:** Verifies that interfaces send and receive accurate data and respond correctly to different inputs.
 - **Example:** Testing API calls to ensure they return correct data formats and handle errors appropriately.
 - **Considerations:** Covers areas like data validation, error handling, and compatibility.
 - **Tools:** Postman and SoapUI are commonly used for interface testing, especially for APIs.
5. **Regression Testing:** Re-runs tests after changes to the code to ensure new updates haven't broken existing functionality.
 - **Scope:** Can cover both individual modules and the entire system, depending on the impact of changes.
 - **Objective:** Confirms that bug fixes, enhancements, or new features haven't introduced new defects.
 - **Types:** Includes full regression testing (for major updates) and selective regression testing (for minor changes).
 - **Frequency:** Performed frequently, especially during continuous integration and deployment.
 - **Tools:** Automation tools like Selenium, QTP, and JUnit are used to streamline regression testing.
6. **User Acceptance Testing (UAT):** Validates that the system meets business requirements and user needs before going live.
 - **Scope:** Focuses on end-to-end workflows to ensure the software behaves as expected from a user's perspective.
 - **Performed By:** Generally conducted by business stakeholders, end-users, or a dedicated UAT team.
 - **Objective:** Ensures that the product fulfills business requirements, is user-friendly, and performs real-world tasks.
 - **Tools:** Tools like Bugzilla and JIRA help manage user feedback and defect tracking during UAT.
 - **Types of UAT:**
 - **Alpha Testing:**
 - Conducted internally by the development team and stakeholders within the organization.

- Allows early feedback and bug detection in a controlled environment.
- Often done in multiple phases to refine functionality and address issues.
- **Beta Testing:**
 - Conducted by a selected group of end-users outside the development team.
 - Exposes the software to real-world usage and gathers feedback on usability, stability, and performance.
 - Beta testers help identify final issues and areas of improvement before the full release.

Non-Functional Testing

1. **Documentation Testing:** Verifies that the software's documentation (user manuals, guides, help files) is accurate, clear, and complete.
 - **Scope:**
 - Ensures instructions for installation, usage, and troubleshooting are user-friendly and informative.
 - Checks for clarity, language accuracy, and ease of understanding for users with different expertise levels.
 - **Importance:** Clear documentation is essential for effective system use, especially for users unfamiliar with the software.
 - **Examples:** Verifying that instructions on how to install the software match the actual installation process, confirming that each feature has an adequate description, and ensuring that error messages are clearly explained.
2. **Installation Testing:** Confirms that the software installs correctly on different platforms, environments, and configurations.
 - **Scope:**
 - Ensures the software installation process completes without errors and meets system compatibility requirements.
 - Checks for dependencies, installation time, configuration options, and uninstallation procedures.
 - **Importance:** A smooth installation process improves the user's initial experience and minimizes technical support needs.
 - **Examples:** Testing installation on various operating systems, verifying that installation completes successfully without errors, and ensuring that all files are correctly placed and accessible after installation.
3. **Performance Testing:** Assesses the software's responsiveness, stability, and scalability under various conditions.
 - **Scope:** Evaluates how the system performs in terms of speed, data processing, resource consumption, and stability under different load conditions.
 - **Importance:** Ensures that the software meets performance benchmarks and can handle peak loads without significant degradation.
 - **Types of Performance Testing:**
 - **Local Testing:**
 - **Objective:** Checks how well the software performs under normal conditions on a local machine or network.
 - **Scope:** Evaluates initial load times, response times, and basic resource consumption.
 - **Examples:** Testing system start-up time and loading of essential functions.
 - **Stress Testing:**

- **Objective:** Determines how the software behaves under extreme loads, pushing beyond normal operational capacity.
 - **Scope:** Tests system robustness by increasing the load until the software fails or becomes unstable.
 - **Examples:** Simulating a sudden spike in traffic on a web application to observe when performance starts to degrade.
 - **Endurance Testing:**
 - **Objective:** Evaluates the software's stability and performance over prolonged usage.
 - **Scope:** Checks for memory leaks, slowdowns, and crashes when the software is used continuously.
 - **Examples:** Running an application for hours or days to monitor if resource usage, like memory, increases unexpectedly.
 - **Spike Testing:**
 - **Objective:** Tests how the software responds to sudden, temporary spikes in usage or load.
 - **Scope:** Observes recovery time and performance stability when the system experiences sharp increases in demand.
 - **Examples:** Simulating multiple simultaneous logins during a flash sale on an e-commerce platform to ensure smooth user experience.
4. **Reliability Testing:** Ensures that the software consistently performs its intended functions under specified conditions over time.
- **Scope:**
 - Focuses on consistency and stability, checking if the software produces expected outcomes without errors during prolonged operation.
 - Often involves testing the system under varying loads to observe how reliability might be affected.
 - **Importance:** Reliability directly impacts user trust and satisfaction, as consistent performance is essential in critical applications.
 - **Examples:** Running automated test scripts multiple times to see if they produce the same result, testing an ATM system for consistent transaction reliability over extended usage.
5. **Security Testing:** Identifies vulnerabilities and ensures that the software is protected against unauthorized access, data breaches, and other security risks.
- **Scope:**
 - Examines various aspects, including data protection, user authentication, and secure communication.
 - Involves testing for common vulnerabilities such as SQL injection, cross-site scripting (XSS), and access control.
 - **Importance:** Security is crucial for safeguarding user data and maintaining trust, especially for applications handling sensitive information.
 - **Examples:** Attempting to bypass login authentication, scanning for potential points of unauthorized access, and verifying that sensitive data, like passwords, are encrypted.

Aspect	Functional Testing	Non-Functional Testing
Objective	Validates actions and behaviours per requirements	Assesses quality and performance characteristics
Focus	Individual functions, features	System-wide performance, reliability, security
Method	Often Black-Box Testing	Black-Box, White-Box, or Gray-Box Testing
Requirements	Driven by functional requirements	Driven by quality and performance requirements
Outcome	Ensures functions work as expected	Ensures user experience, efficiency, and resilience
Tools	Selenium, QTP, JUnit	JMeter, LoadRunner, AppDynamics, Burp Suite

Testing Strategies and Levels

1. Black-box Testing: Black-box testing is a method where the tester focuses solely on the **functional requirements** of the software without any knowledge of its internal workings or code structure. The tester does not need to know how the software is designed or how it performs its tasks. Instead, the focus is on the **input-output relationships**—providing inputs and checking whether the outputs meet the expected results.

- **Focus:** Functionality, user experience, and system behaviour.
- **What the tester does:** Tests the software by providing inputs and verifying whether the outputs meet the functional requirements.
- **Internal workings are irrelevant:** The internal logic, algorithms, or code structure are not examined during black-box testing.
- **Types of black-box testing:**
 - **Functional Testing:** Validating that the software performs the intended functions.
 - **Non-functional Testing:** Checking aspects such as performance, security, and usability.
 - **Boundary Value Testing:** Checking inputs at the edge of acceptable ranges.
 - **Equivalence Partitioning:** Dividing input data into valid and invalid groups for more efficient testing.
- **Advantages:**
 - The tester doesn't need to know the software's internal code, making it easier to test from a user's perspective.
 - Useful for validating high-level functionality against requirements.
 - Focuses on the system's behaviour and ensures that it meets user expectations.
- **Disadvantages:**
 - May miss potential errors in the system's internal processes and logic.
 - Limited in covering all possible code paths and execution scenarios.
- **Example:** For a login system, a tester inputs different combinations of usernames and passwords to check if the system behaves correctly (i.e., accepting valid credentials and rejecting invalid ones).

2. White-box Testing: White-box testing, also known as **clear-box** or **structural testing**, involves testing the **internal structure** and **logic** of the software. The tester has full access to the code and architecture and tests the system's behaviour based on this detailed knowledge. This method helps ensure that the internal logic, code paths, and structures work as intended.

- **Focus:** Code structure, logic, algorithms, and paths.
- **What the tester does:** Tests individual parts of the system based on the code (such as functions, methods, or branches in the logic). The goal is to ensure that the code executes correctly in different scenarios, covering all possible code paths.
- **Internal workings are essential:** The tester needs a deep understanding of the code, architecture, and design of the software.
- **Types of white-box testing:**
 - **Unit Testing:** Testing individual units or components of the code to ensure they function correctly.
 - **Code Coverage:** Ensuring that all code paths (including if-else conditions) are tested.
 - **Path Testing:** Testing all possible execution paths in the code to ensure correctness.
 - **Loop Testing:** Checking the correctness of loops in the code.
- **Advantages:**
 - Helps identify hidden errors or issues in the internal logic and structure of the software.
 - Ensures that all code paths and branches are covered, improving test coverage.
 - Effective in detecting problems related to code, algorithms, and performance.
- **Disadvantages:**
 - Requires in-depth knowledge of the code and can be time-consuming.
 - Not suitable for testing the software from an end-user's perspective.
- **Example:** For a login system, a tester would check the function handling user authentication, inspecting whether all code paths for successful and unsuccessful logins are executed correctly.

3. Gray-box Testing: Gray-box testing is a hybrid approach that combines elements of both black-box and white-box testing. In gray-box testing, the tester has partial knowledge of the internal workings of the software but does not have complete access to the source code. This approach often uses a **limited understanding of the system's internals** to design tests while still focusing primarily on testing the software from the user's perspective.

- **Focus:** Combination of functionality and internal structure.
- **What the tester does:** The tester uses knowledge of the system's architecture, design, or database (but not the full code) to test the software. They might focus on areas such as security, session management, or application flows that involve internal systems.
- **Internal workings are partially known:** The tester might have access to high-level design or architecture documentation, but not the complete codebase. This allows them to create more targeted test cases than with black-box testing.
- **Advantages:**
 - More targeted than black-box testing because the tester has partial knowledge of the system.
 - Combines the best of both worlds by addressing both functional aspects (like black-box) and internal structure (like white-box).
 - More effective for testing system security, data handling, and integrations with other systems.
- **Disadvantages:**

- Limited by the tester's partial knowledge of the system, which may result in missing certain defects or code issues.
- Still requires careful design and planning to ensure proper coverage.
- **Example:** In the case of a web application, a tester might use knowledge of the backend database or server architecture to test for security vulnerabilities, such as SQL injection or data leakage, while still interacting with the application as a user would.

Testing Type	Focus	Internal Knowledge Required	Testing Approach	Example
Black-box Testing	Functionality, inputs, outputs	None	Tests software without knowledge of its code	Testing the login system with valid/invalid credentials
White-box Testing	Code structure, internal logic	Full knowledge	Tests the internal structure and logic	Testing the user authentication code for edge cases
Gray-box Testing	Combination of functionality and internal workings	Partial knowledge	Tests the system using partial understanding of the internals	Testing security vulnerabilities in a web application with partial backend knowledge

Reliability Assessment

- **Objective:** Measures the software's ability to function consistently without failure over time.
- **Scope:**
 - Evaluates how likely the software is to perform correctly and reliably under specified conditions.
 - Helps predict failure rates, system availability, and operational robustness.
- **Methods:**
 - **Reliability Testing:** Uses stress and endurance tests to simulate long-term usage.
 - **Failure Analysis:** Studies failure patterns to determine root causes and improve reliability.
- **Importance:** Essential for mission-critical applications, as reliability directly impacts user trust and safety.
- **Examples:** Running reliability tests on a banking system to ensure it can process transactions without error over extended usage.

Validation & Verification (V&V)

- **Objective:** Ensures that the software meets all specifications (Verification) and fulfils its intended purpose (Validation).
- **Scope:**
 - **Verification:** Confirms that the software was built correctly according to design specifications.
 - **Validation:** Confirms that the right software was built, meeting user requirements and expectations.
- **Methods:**

- **Verification Techniques:** Includes reviews, inspections, static analysis, and white-box testing.
- **Validation Techniques:** Involves dynamic testing, user acceptance testing, and black-box testing.
- **Importance:** Vital for identifying defects early in development, saving time and costs by reducing post-release issues.
- **Examples:** Verification through code reviews to ensure all modules are correctly implemented; validation by running user acceptance tests with real-world scenarios.

12 Key Metrics in Software Testing

1. **Defect Density:** Number of defects per unit of code (e.g., defects per 1,000 lines of code), indicating code quality.
2. **Defect Removal Efficiency (DRE):** Percentage of defects removed during development versus those found after release, reflecting testing effectiveness.
3. **Mean Time to Failure (MTTF):** Average time the system operates before a failure, indicating reliability.
4. **Mean Time to Repair (MTTR):** Average time required to fix a defect, showing maintainability.
5. **Mean Time Between Failures (MTBF):** Average time between successive failures, indicating stability.
6. **Test Case Effectiveness:** Ratio of passed test cases to total test cases, measuring test efficiency.
7. **Test Coverage:** Percentage of code covered by tests, showing the extent of testing.
8. **Test Execution Time:** Total time taken to execute all test cases, indicating testing efficiency.
9. **Defect Leakage:** Number of defects missed during testing but found after release, measuring testing thoroughness.
10. **Escaped Defects:** Defects detected after release, showing areas needing improvement.
11. **Test Automation Coverage:** Ratio of automated tests to total tests, indicating automation progress.
12. **Customer-Reported Defects:** Defects reported by user's post-release, a direct reflection of software quality.

Monitoring & Control in Software Testing

- **Objective:** Provides ongoing oversight of testing progress and effectiveness, allowing for adjustments to ensure quality targets are met.
- **Scope:**
 1. **Tracking Metrics:** Regularly monitor metrics (like defect density, test coverage) to identify areas for improvement.
 2. **Status Reporting:** Periodic reports on test progress, defect resolution, and test coverage help stakeholders understand testing effectiveness.
 3. **Quality Control:** Real-time monitoring identifies any deviations from quality standards, prompting corrective actions.
 4. **Risk Management:** By continuously assessing risks, teams can prioritize tests and allocate resources efficiently.
- **Methods:**
 1. **Test Management Tools:** Tools like JIRA, TestRail, or HP ALM track testing progress and defects.

2. **Dashboards:** Visual representations of metrics and progress make it easier for stakeholders to track project health.
 3. **Feedback Loops:** Regular feedback from test results and stakeholder input supports iterative improvements.
- **Importance:** Enables teams to deliver reliable, high-quality software on time and within budget by proactively addressing issues.
 - **Examples:** Using dashboards to monitor defect trends over time, providing insight into areas needing more testing or code quality improvements.

MODULE 4: SOFTWARE PROJECT MANAGEMENT

Contents

1. Project Scheduling	1
2. Staffing in Software Development Projects	2
3. Software Configuration Management (SCM).....	3
4. Quality Assurance (QA)	5
5. Project Monitoring	5
6. Software Reliability Management.....	6
7. Risk Management in Software Development	7
8. ECASE Tools (Extended CASE Tools).....	9
9. Team Structure of an Organization	9

Project Scheduling

Project scheduling is a critical aspect of project management that involves determining the timeline and sequence of tasks required to complete a project within defined constraints like time, resources, and cost. It includes creating a detailed plan outlining when and how activities should be performed to achieve project goals. **Key Steps in Project Scheduling**

1. **Define Activities:** Break down the project into manageable tasks derived from the Work Breakdown Structure (WBS).
2. **Sequence Activities:** Identify task dependencies to determine task order and logical relationships.
3. **Estimate Resources:** Identify required resources (personnel, equipment, materials) for each task.
4. **Estimate Durations:** Predict task durations using techniques like expert judgment and three-point estimation.
5. **Develop the Schedule:** Create a project timeline, often visualized in a Gantt chart or network diagram.
6. **Determine Critical Path:** Identify the sequence of tasks that directly impacts project completion.
7. **Adjust and Optimize the Schedule:** Refine the schedule using methods like resource leveling or crashing to improve efficiency.
8. **Set Milestones:** Define key points to mark significant project phases or deliverables.

A. Scheduling Tools and Techniques

1. **Gantt Chart:** A visual representation of the project schedule where tasks are represented as horizontal bars along a timeline. Gantt charts are used to track task start and end dates and their progress.
2. **Network Diagrams:** A graphical representation of tasks and dependencies, showing how tasks are connected. Network diagrams help identify the critical path and are used in methods like CPM and PERT.

3. **Critical Path Method (CPM):** A scheduling technique used to identify the sequence of critical tasks and calculate the minimum time needed to complete the project. It helps focus on tasks that must be completed on time.
4. **Program Evaluation and Review Technique (PERT):** A statistical approach to scheduling that uses three-time estimates (optimistic, pessimistic, and most likely) for each task. PERT helps in managing uncertainty in time estimates.
5. **Resource Levelling:** A technique used to resolve over-allocation of resources. It adjusts the start and finish dates of tasks to ensure that resources are not overbooked or underused.
6. **Fast-Tracking and Crashing:** **Fast-Tracking** involves overlapping tasks that were originally planned to be done sequentially, to reduce project duration.
7. **Crashing** involves adding extra resources to critical path tasks to reduce their duration.

B. Importance of Project Scheduling

1. **Ensures Timely Completion:** By clearly defining task durations and deadlines, scheduling helps ensure that the project is completed on time.
2. **Resource Management:** Proper scheduling helps optimize resource allocation, preventing underutilization or overloading of resources.
3. **Improves Cost Efficiency:** By minimizing delays and ensuring smooth task transitions, project scheduling helps keep the project within budget.
4. **Facilitates Risk Management:** Scheduling helps identify potential bottlenecks and dependencies early, allowing the team to mitigate risks.
5. **Tracks Progress:** With clear milestones and deadlines, the project team can easily track progress and adjust when necessary.

C. Challenges in Project Scheduling

1. **Uncertainty in Time Estimates:** It is difficult to accurately predict how long each task will take, especially in complex projects.
2. **Resource Constraints:** Limited availability of resources can delay tasks, especially if they are shared across multiple projects.
3. **Changing Requirements:** Modifications in scope or requirements during the project can disrupt the established schedule.
4. **External Dependencies:** Dependencies on external vendors, clients, or other teams can introduce delays that are beyond the project team's control.

Staffing in Software Development Projects

Staffing refers to the process of identifying, hiring, and managing the human resources needed to complete a software development project. It involves planning, recruiting, onboarding, managing, and retaining team members, ensuring that the project has the right skills, expertise, and manpower at each phase. **Key Components of Staffing**

1. **Resource Planning:** Identifying the roles and skills required for the project based on project scope and technical requirements. Creating a project organization chart with details on team composition (e.g., project managers, developers, testers, analysts, etc.).
2. **Recruitment:** Hiring skilled personnel to meet the project's needs. Recruitment may involve internal transfers, external hires, or contracting. Ensuring a mix of skills (e.g., technical, domain expertise, communication) to handle different tasks.
3. **Onboarding:** Introducing new team members to the project, tools, processes, and goals. Training and mentoring new staff to ensure they are productive quickly.
4. **Managing the Team:**
 - Assigning tasks and responsibilities based on team members' skills.
 - Monitoring team performance and adjusting workload to balance efficiency and morale.
 - Encouraging collaboration and addressing conflicts or performance issues.
5. **Staff Retention:**
 - Providing a conducive work environment, growth opportunities, and rewards to retain top talent.
 - Reducing turnover to maintain project continuity and reduce onboarding costs.

Challenges in Staffing:

1. **Skill Gaps:** Difficulty finding the right talent, especially for specialized roles.
2. **Overstaffing/Understaffing:** Imbalance in resource allocation can affect productivity and project deadlines.
3. **Team Dynamics:** Managing interpersonal conflicts and maintaining motivation can be challenging.

Software Configuration Management (SCM)

Software Configuration Management (SCM) is the process of systematically controlling changes to the software to maintain integrity and traceability throughout the software development lifecycle. SCM ensures that the software evolves in a controlled manner, preventing issues like conflicting updates or loss of important changes. **Key Components of SCM**

1. **Version:** A specific state of the software that includes all updates, changes, and new features made up to a certain point. Versions are typically assigned numbers (e.g., 1.0, 2.0) to track major developments.
2. **Revision:** Refers to **minor updates** or corrections within a version, often related to bug fixes or small improvements. Revisions are generally denoted with additional numbers (e.g., 1.1, 1.2).
3. **Release:** The process of distributing a **version** of the software to users or customers. A release can include both new features and fixes, and it marks the point when the software is made available outside the development team.
4. **Version Control:**
 - Tracks and manages changes to the codebase over time. Tools like Git, SVN, or Mercurial are used to manage versions and allow multiple developers to collaborate on the same codebase without conflict.

- **Branching and Merging:** Creating branches to work on features or bug fixes separately and then merging them into the main codebase after validation.

5. **Baseline Management:**

- A **baseline** is a stable version of the software that can be used as a reference point. Baselines are created at specific milestones (e.g., after testing) and are used for comparison when further changes are made.
- Ensures that the software can be reverted to a stable state if issues arise during development.

6. **Change Control:**

- Establishing procedures to approve, track, and manage changes in the software. Change requests are reviewed and evaluated for their impact before being incorporated.
- Change control boards (CCBs) may be used to oversee and approve changes to the software.

7. **Build Management:**

- Automating the process of compiling, linking, and packaging code to create an executable application. Continuous Integration (CI) tools like Jenkins or Travis CI are used to automate builds and tests.
- Ensures that software can be consistently and reliably built, minimizing errors during deployment.

8. **Configuration Audits:**

- Regular reviews or audits are conducted to ensure that the configuration of the software aligns with the approved baselines and that no unauthorized changes have been made.
- Audits verify that the software remains compliant with organizational or project standards.

9. **Release Management:**

- Coordinating the deployment of the software from development to production environments. SCM helps track which versions are deployed, manage release notes, and ensure smooth transitions between versions.
- Ensures that releases are planned, tested, and delivered in a structured manner, reducing risks.

Importance of SCM

1. **Consistency and Control:** Ensures that software development follows a structured process, preventing unauthorized changes or conflicts in the codebase.
2. **Traceability:** Every change is tracked, allowing developers to understand the history of the project and trace bugs back to specific changes.
3. **Collaboration:** Supports multiple developers working on the same codebase, preventing conflicts through effective version control and merging.
4. **Quality Assurance:** By managing changes systematically, SCM helps maintain software quality and ensure stability through controlled builds and releases.

Common SCM Tools

1. **Git:** A distributed version control system widely used for managing code repositories.
 2. **Subversion (SVN):** A centralized version control system.
 3. **Jenkins:** A continuous integration tool that automates builds and testing.
 4. **Bitbucket/GitHub/GitLab:** Platforms for hosting Git repositories with added collaboration and management features.
-

Quality Assurance (QA)

Quality Assurance (QA) in software development refers to the systematic processes and activities undertaken to ensure that the software meets the specified requirements and is free of defects. QA is proactive, focusing on preventing issues during the software development process, rather than just finding them after testing. **Key Components of QA**

1. **Standards and Procedures:** Defining clear standards, processes, and methodologies to ensure quality throughout the development lifecycle. This includes guidelines for coding, documentation, design, and testing.
2. **Verification and Validation:**
 - **Verification:** Ensures that the software is built according to the requirements and design specifications (e.g., code reviews, design reviews).
 - **Validation:** Ensures that the software fulfils the intended purpose and meets user needs (e.g., functional testing, user acceptance testing).
3. **Testing:** Involves different types of testing such as unit testing, integration testing, system testing, and acceptance testing to detect defects.
4. **Defect Management:** Tracking and managing bugs and issues found during testing to ensure they are resolved before the software is released.
5. **Continuous Improvement:** Implementing a feedback loop to continuously improve processes and methodologies based on previous experiences and outcomes (e.g., through retrospectives, root cause analysis).

Importance of QA:

- **Prevents Defects:** By implementing quality checks early, QA helps prevent defects from entering the later stages of development.
 - **Customer Satisfaction:** Ensures that the final product meets customer expectations and performs as intended.
 - **Cost Savings:** Catching defects early in the development process reduces the cost of fixing them later.
-

Project Monitoring

Project Monitoring involves tracking the progress of a software development project to ensure it stays on schedule, within budget, and meets its objectives. It helps project managers keep an eye on

key performance indicators (KPIs) to identify potential problems early and adjust as needed. **Key Components of Project Monitoring**

1. **Progress Tracking:** Monitoring the completion of tasks and milestones as defined in the project schedule. Tools like Gantt charts and Kanban boards are commonly used to visualize progress.
2. **Resource Management:** Tracking resource utilization to ensure that human, financial, and material resources are allocated and used efficiently.
3. **Risk Management:** Identifying, tracking, and mitigating risks as they arise during the project lifecycle. Regular risk reviews are important to prevent issues from escalating.
4. **Cost Monitoring:** Keeping track of project costs to ensure that the project remains within budget. Variances from the planned budget are analysed, and corrective actions are taken if needed.
5. **Schedule Monitoring:** Ensuring that the project timeline is being followed. Critical Path Method (CPM) or Program Evaluation and Review Technique (PERT) can be used to identify potential delays.
6. **Performance Metrics:** Establishing KPIs such as defect density, completion rate, and resource efficiency to track the overall health of the project.
7. **Reporting and Communication:** Regularly reporting the project status to stakeholders, including updates on progress, risks, issues, and changes.

Importance of Project Monitoring:

- **Ensures Timely Delivery:** Helps ensure that the project stays on track and is delivered within the agreed-upon time frame.
- **Enhances Decision-Making:** Provides real-time data that helps managers make informed decisions.
- **Reduces Risks:** Early identification of problems or deviations from the plan allows for timely corrective actions.

Software Reliability Management

Software Reliability Management refers to the activities and processes aimed at ensuring that the software performs consistently and predictably under specific conditions for a defined period. It involves both predicting and improving the reliability of software systems. **Key Components of Software Reliability Management**

1. **Fault Tolerance:** Techniques such as redundancy and error-handling mechanisms are implemented to ensure the system continues to function even when components fail.
2. **Reliability Testing:**
 - Includes stress testing, load testing, and fault-injection testing to assess how the system performs under extreme or error-prone conditions.
 - **Regression Testing:** Ensures that previously developed and tested software still performs after a change.

3. **Failure Tracking:** Recording and analysing software failures to identify trends, patterns, or recurring defects. Failure tracking systems (e.g., JIRA) are used to log defects and track their resolution.
4. **Root Cause Analysis:** Investigating the root cause of failures and implementing corrective actions to prevent similar issues from occurring in the future.
5. **Reliability Growth Models:** Models like the **Jelinski-Moranda** or **Musa Model** predict the reliability growth of the software as faults are discovered and fixed during testing.
6. **Software Reliability Metrics:**
 - i. **ROCOF (Rate of Occurrence of Failures):** Measures the frequency of software failures per unit of time. It indicates how often the software fails during operation. $ROCOF = \frac{\text{Number of Failures}}{\text{Total Time of Operation}}$.
 - ii. **MTTF (Mean Time to Failure):** The average time the software operates before a failure occurs. Higher MTTF indicates better reliability. $MTTF = \frac{\text{Total Operational Time}}{\text{Number of Failures}}$.
 - iii. **MTTR (Mean Time to Repair):** The average time taken to repair the software after a failure, including diagnosis and correction time. $MTTR = \frac{\text{Total Repair Time}}{\text{Number of Failures}}$.
 - iv. **MTBF (Mean Time Between Failures):** The average time between consecutive failures, calculated as the sum of MTTF and MTTR. It reflects the system's overall reliability. $MTBF = MTTF + MTTR$.
 - v. **POFO (Probability of Failure on Demand):** The probability that the software will fail when a demand for service occurs. It's used in safety-critical systems. $POFO = \frac{\text{Number of Failures}}{\text{Number of Demands}}$.
 - vi. **Availability:** Measures the proportion of time the software is operational and available for use. It depends on both MTBF and MTTR. $Availability = \frac{MTBF}{MTBF + MTTR}$.

Importance of Software Reliability Management:

- **Minimizes Downtime:** Ensures that the software is available and functional, minimizing unplanned outages and system crashes.
- **User Trust:** Reliable software builds trust with users, as it reduces the likelihood of errors or system failures during operation.
- **Cost Efficiency:** Reliable systems require fewer repairs, reducing long-term maintenance costs.

Risk Management in Software Development

Risk Management involves identifying, assessing, and mitigating risks that could potentially impact a software development project. Risks can arise from a variety of sources, including technical challenges, changing requirements, resource constraints, or external factors like market or regulatory changes. **Steps in Risk Management**

1. Risk Identification:

- Identifying potential risks that may affect the project (e.g., delays in delivery, changes in technology, budget overruns).
- Types of risks include technical risks, operational risks, market risks, and legal risks.

2. Risk Assessment:

- **Probability:** Determining how likely each risk is to occur.
- **Impact:** Assessing the severity of the consequences if the risk occurs (e.g., project delay, cost increase).
- Risks are often categorized as low, medium, or high based on their probability and impact.

3. Risk Prioritization:

Prioritizing risks based on their potential impact and likelihood. Focus is placed on high-priority risks that could have significant effects on the project.

4. Risk Mitigation:

Developing strategies to reduce the likelihood or impact of risks. Mitigation strategies can include adding contingency plans, building in buffers for time and resources, or implementing risk-reducing technologies.

5. Risk Monitoring:

Continuously tracking risks throughout the project lifecycle, reassessing them, and implementing mitigation strategies as needed.

6. Contingency Planning:

Preparing backup plans or alternative approaches if high-priority risks do occur. This helps in minimizing disruption to the project.

7. Risk Accessing:

$p = r * s$, p = priority of each risk, r = probability of risk being true, s = consequent problems associated with the risk.

8. Types of risk

- **Project Risk:** Refers to uncertainties that may impact the overall success of a project, including scope, time, cost, and quality. It includes risks like resource constraints, schedule delays, and miscommunication within the team.
- **Technical Risk:** Involves challenges related to the **technology** used in the project, such as software defects, hardware failures, integration issues, or technological limitations that could affect functionality or performance.
- **Business Risk:** Relates to the potential **impact** on the business, such as financial loss, market changes, or shifting customer requirements. These risks can affect the project's return on investment, profitability, or competitive advantage.

Importance of Risk Management:

- **Reduces Uncertainty:** Helps teams anticipate potential issues, which leads to better planning and fewer surprises.
- **Improves Decision-Making:** Knowing the risks helps in making informed decisions and adjusting the project plan accordingly.
- **Minimizes Losses:** Proper mitigation can reduce project delays, budget overruns, and scope creep.

ECASE Tools (Extended CASE Tools)

CASE (Computer-Aided Software Engineering) tools support software development by automating tasks such as code generation, testing, and design modelling. **ECASE (Extended CASE) tools** take it a step further by integrating advanced project management, documentation, and collaboration features.

Types of CASE Tools:

- **Upper CASE Tools:** Assist in early stages like requirements gathering, design modelling, and planning (e.g., UML modelling tools).
- **Lower CASE Tools:** Support later stages such as coding, testing, and maintenance (e.g., IDEs, testing frameworks).
- **Integrated CASE (I-CASE):** Combines both upper- and lower-case functionalities for end-to-end software lifecycle support.

Advantages of ECASE Tools:

- **Improved Productivity:** Automates repetitive tasks, reducing manual effort.
 - **Standardization:** Enforces coding, design, and documentation standards across teams.
 - **Better Collaboration:** Facilitates communication and coordination among distributed teams.
 - **Risk Reduction:** Helps in identifying potential issues early in the project lifecycle.
 - **Documentation:** Automatically generates documentation, making it easier to track project progress.
-

Team Structure of an Organization

The structure of a software development team within an organization can vary depending on the project's size, complexity, and organizational goals. The most common team structures are.

1. Functional Team Structure: Team members are grouped based on their specialized functions or skills (e.g., developers, testers, designers).

- **Advantages:**
 - Clear hierarchy and specialization.
 - Team members gain deep expertise in their respective fields.
- **Disadvantages:**
 - Limited cross-functional communication, which may lead to slower decision-making.
 - Siloed thinking that can reduce innovation.

2. Project-Based Team Structure: Teams are organized around specific projects, with members from different functional areas assigned to work on the project full-time.

- **Advantages:**
 - High focus on the project, with faster decision-making.
 - Clear accountability and ownership of deliverables.
- **Disadvantages:**
 - Once the project is over, team members may need to be reassigned.
 - Resource redundancy, as multiple teams may need similar expertise.

3. Matrix Team Structure: A hybrid of functional and project-based structures. Team members report to both a functional manager and a project manager.

- **Advantages:**
 - Flexible resource allocation, allowing members to contribute to multiple projects.
 - Encourages collaboration between functional and project teams.
- **Disadvantages:**
 - Dual reporting can lead to confusion or conflict.
 - Resource juggling can create overload and reduce focus.

4. Chief Programming Team: A **centralized** team structure where one **chief programmer** acts as the leader, responsible for making major design decisions and overseeing the project. Other team members assist by handling coding and routine tasks.

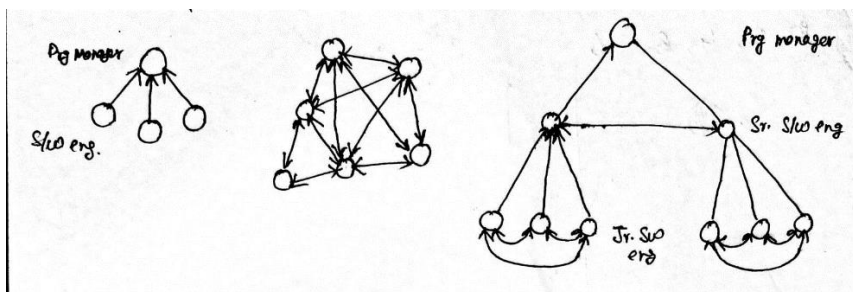
- **Advantages:** Efficient decision-making, strong leadership, and high accountability.
- **Disadvantages:** Bottlenecks can occur if the chief is overburdened or unavailable.

5. Democratic Programming Team: A **decentralized** structure where all team members have an equal say in decision-making. The team works collaboratively, and leadership is **shared**.

- **Advantages:** Promotes creativity and collaboration, fostering better team engagement.
- **Disadvantages:** Can lead to slower decisions and potential conflicts due to differing opinions.

6. Mixed Programming Team: Combines aspects of both the **chief** and **democratic** structures. A **chief programmer** provides guidance, but decisions are made with input from the whole team.

- **Advantages:** Balances strong leadership with team collaboration.
- **Disadvantages:** Requires careful management to avoid conflicts between the leader and the team members.



A. Roles in a Software Development Team:

1. **Project Manager:** Oversees the project's execution, manages the schedule, and ensures that the project meets its goals.
2. **Software Architect:** Designs the overall system architecture, ensuring it aligns with both technical and business requirements.
3. **Developers:** Responsible for writing code, implementing features, and maintaining the software system.
4. **Testers/QA Engineers:** Ensure the software product is free of bugs and meets quality standards through rigorous testing and validation.
5. **Business Analysts:** Liaise between stakeholders and the development team to ensure requirements are accurately captured and met.
6. **UI/UX Designers:** Create intuitive and user-friendly designs for the software interface and user experience.

7. **DevOps Engineers:** Manage deployment pipelines, automation, and infrastructure to ensure smooth integration and delivery.

B. Roles of a Project Manager:

1. **Planning:** Develops the overall project plan, setting objectives, timelines, resources, and budgets.
2. **Task Assignment:** Delegates tasks to team members based on their skills and availability, ensuring balanced workloads.
3. **Risk Management:** Identifies potential risks and develops strategies to mitigate them, ensuring the project stays on track.
4. **Communication:** Acts as the main point of contact between stakeholders, clients, and the project team, ensuring clear and consistent communication.
5. **Monitoring and Control:** Tracks project progress, monitors timelines and budgets, and adjusts as needed to keep the project aligned with its goals.
6. **Quality Assurance:** Ensures that the project deliverables meet quality standards and client expectations.
7. **Problem Solving:** Addresses any issues or roadblocks that arise during the project, facilitating timely solutions.
8. **Team Leadership:** Motivates and supports the project team, ensuring collaboration and maintaining morale.
9. **Stakeholder Management:** Manages relationships with stakeholders to align the project with business objectives and user needs.
10. **Project Closure:** Ensures proper documentation, assesses project outcomes, and ensures a smooth handover upon completion.

C. Characteristics of a Good Software Engineer:

1. **Strong Problem-Solving Skills:** Ability to analyse complex issues and develop efficient, innovative solutions.
2. **Proficiency in Programming:** Mastery of various programming languages, tools, and frameworks, with a focus on writing clean, maintainable code.
3. **Attention to Detail:** Ensures accuracy and precision in coding, debugging, and testing to minimize errors.
4. **Adaptability:** Willingness to learn new technologies and adapt to evolving project requirements.
5. **Effective Communication:** Ability to convey technical information clearly to both technical and non-technical stakeholders.
6. **Team Collaboration:** Works well in a team environment, contributing ideas and collaborating on tasks with other team members.
7. **Time Management:** Efficiently manages time to meet deadlines, prioritizing tasks and balancing workloads effectively.
8. **Creativity:** Innovates and approaches problems with new perspectives, thinking outside the box when needed.
9. **Ethical Mindset:** Upholds ethical standards, ensuring the software is reliable, secure, and used responsibly.
10. **Continuous Learning:** Actively seeks to improve skills and stay updated with the latest industry trends and best practices.

MODULE 4: STATIC & DYNAMIC MODELS

Contents

1. Models	1
2. UML	2
3. Static Models.....	3
4. Dynamic Models	4
5. State and Activity Models	5
6. Object-Oriented Modelling (Classes, Objects, and Messages)	6

Models

Modelling in software engineering refers to the process of creating abstract representations of a system. These representations, or "models," help software engineers understand, design, and document the system's structure, behaviour, and interactions. Models are used to simplify complex systems by focusing on key aspects while leaving out unnecessary details.

1. Purpose of Modelling:

- **Simplification:** Models reduce the complexity of real-world systems, making it easier to understand and manage them.
- **Communication:** Models provide a common language for developers, stakeholders, and analysts, aiding in clearer communication.
- **Documentation:** Models serve as documentation for the system's design, behaviour, and structure, which helps future developers and maintainers.
- **Analysis and Testing:** Models allow for the simulation of system behaviour, enabling analysis and testing of various scenarios before actual implementation.

2. Types of Models:

In software engineering, models are categorized based on their focus: **Static Models** and **Dynamic Models**.

i. Static Models:

- Focus on the structure of a system.
- Represent elements like objects, classes, components, and their relationships.
- Typically depict things that do not change over time, such as the system's architecture or data structure.
- Examples include class diagrams, entity-relationship diagrams, and component diagrams.

ii. Dynamic Models:

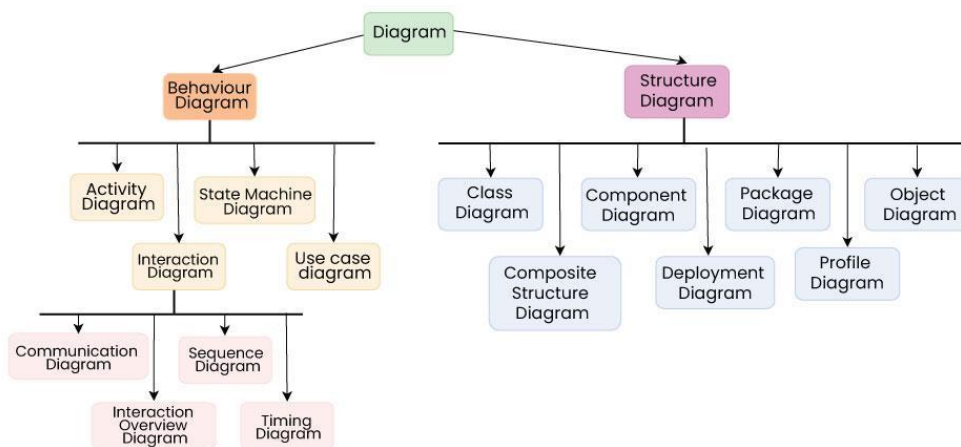
- Focus on the behaviour of a system over time.
- Represent how a system reacts to external and internal events, changes in state, and the interactions between system components.
- Examples include state diagrams, sequence diagrams, activity diagrams, and communication diagrams.

UML

In software engineering, UML (Unified Modelling Language) is a standardized visual language used to specify, visualize, construct, and document the components of software systems. It helps developers and stakeholders understand and communicate complex system designs by providing a structured way to create diagrams representing the system's structure and behaviour. Types of UML:

A. Behavioural Diagrams: Behavioural diagrams describe the dynamic behaviour of the system, capturing the interactions, activities, and state transitions.

1. **Activity Diagram:** Visualizes workflows and the sequence of activities within a process or operation.
2. **Interaction Diagrams**
 - **Communication Diagram:** Illustrates interactions between objects or components, showing their relationships and message flows.
 - **Interaction Overview Diagram:** Combines activity and interaction diagrams, showing a high-level control flow of interactions.
 - **Sequence Diagram:** Depicts the order of messages exchanged between objects over time in a specific scenario.
 - **Timing Diagram:** Shows the behaviour of objects over a time, focusing on timing constraints and object states.
3. **State Machine Diagram:** Captures the states of an object and transitions between states based on events, useful for systems with significant state-dependent behaviour.
4. **Use Case Diagram:** Models user interactions, representing the relationships between actors and use cases that define system functionalities.



UML Diagrams



B. Structural Diagrams: Structural diagrams focus on the static structure of the system, showing system components and their relationships.

1. **Class Diagram:** Defines the structure of classes, their attributes, methods, and relationships, providing a blueprint of the system's classes.
2. **Component Diagram:** Illustrates physical components (such as modules and libraries) and their dependencies, commonly used for implementation and deployment views.
3. **Object Diagram:** Represents instances of classes and the relationships between them at a particular moment, providing a snapshot of the system state.
4. **Package Diagram:** Groups related elements into packages, showing their organization and dependencies, often used for organizing the system structure.
5. **Profile Diagram:** Allows for customization of UML models for specific domains by defining custom stereotypes, tagged values, and constraints.

Aspect	Behavioural Diagrams	Structural Diagrams
Purpose	Capture dynamic behaviour and interactions within the system.	Represent static aspects and organization of the system.
Focus	Processes, workflows, interactions, and state changes over time.	System elements and their relationships, structure, and hierarchy.
What it Models	How the system behaves and responds to events.	How the system is organized and built structurally.
Examples	Activity Diagram, Sequence Diagram, Use Case Diagram, State Machine Diagram	Class Diagram, Component Diagram, Object Diagram, Package Diagram
System View	Shows " what the system does " in terms of actions and interactions.	Shows " how the system is built " and its organizational structure.
Key Elements	Objects, actions, messages, states, transitions.	Classes, components, packages, relationships.
Temporal Focus	Models time-based changes and responses to specific events or actions.	Models static structure that generally remains consistent over time.
Use Cases	Useful for visualizing workflows, state transitions, and user-system interactions.	Useful for visualizing class hierarchies, component dependencies, etc.
Diagram Type	Dynamic — emphasizes system behaviour over time	Static — emphasizes system organization
Complementary Nature	Shows interaction flow during execution	Supports behavioural diagrams by detailing structural dependencies

Static Models

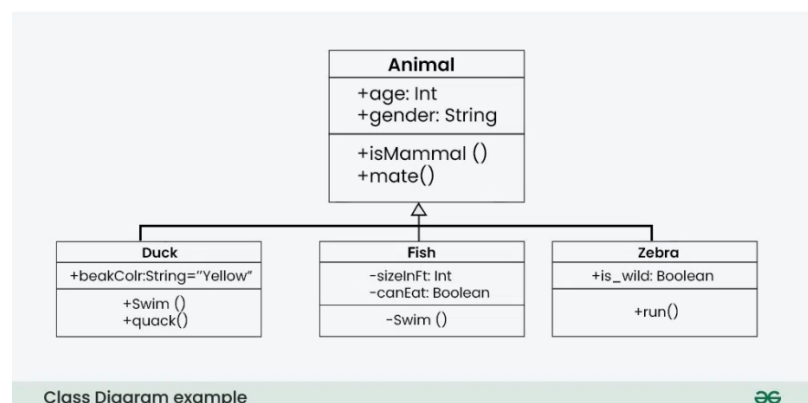
Static models in software engineering are used to represent the **structure** of a system. These models focus on aspects that remain unchanged during execution, such as components, relationships, data, and attributes. Static models provide a blueprint for the system's design, showing how the various parts of the system are organized and interact with each other at a high level. **Key Concepts:**

1. Purpose:

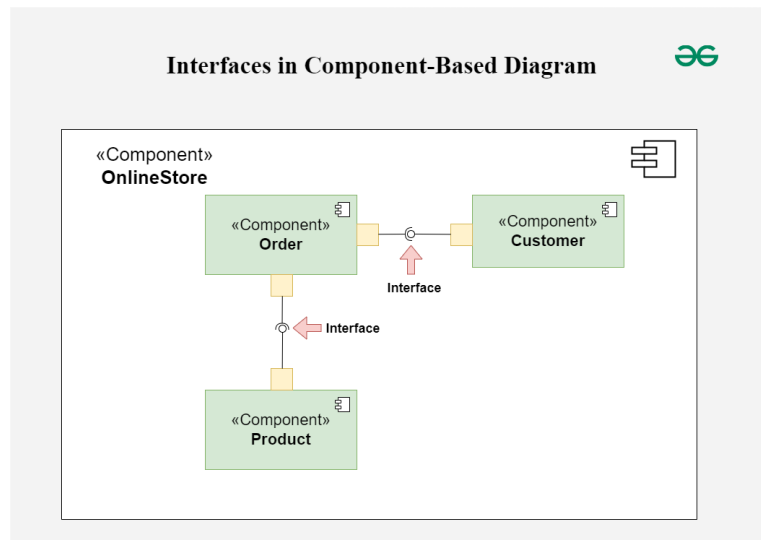
- To **define system components** and their relationships.
- To **structure the system**, ensuring that components are organized in a way that supports easy modification, maintenance, and scalability.
- To **document design decisions** for clarity and understanding across teams.

2. Types of Static Models:

- **Class Diagrams (Object-Oriented Modelling):** A class diagram is one of the most common types of static models in object-oriented systems. It shows the **classes** in the system, their **attributes**, and their **methods**, along with the relationships between classes (inheritance, association, etc.).



- **Component Diagrams:** These diagrams show how **components** in the system interact with each other. A component is a modular part of the system that can be independently replaced, tested, or maintained. The component diagram shows the **interfaces** and **dependencies** between components.



- **Data Flow Diagrams (DFD):** DFDs represent the flow of data within the system. They show how **data inputs** are processed to yield outputs. DFDs highlight the **processes**, **data stores**, and **external entities** that interact with the system.

3. Benefits of Static Models:

- Help to visualize the system's architecture early in the design process.
- Simplify the identification of system components and their relationships.
- Serve as a guide for developers during the coding phase.

Dynamic Models

Dynamic models focus on the **behaviour** of a system over time. These models are used to depict how the system responds to inputs, state changes, or external events during its execution. Dynamic models show how the system behaves & evolves representing the changing aspects of the system, such as **state transitions**, **method calls**, and **interactions** between system components. They describe how the system reacts to specific actions or inputs and how data or states change over time **Key Concepts:**

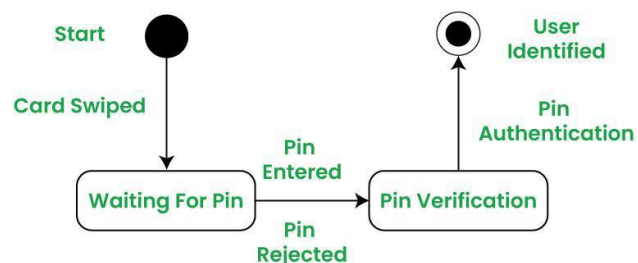
1. Purpose:

- To describe **system behaviour** in terms of inputs, outputs, and internal state changes.
- To simulate and understand the **dynamic interactions** within the system.
- To validate that the system's design will fulfil the requirements under various conditions.

2. Types of Dynamic Models:

- **State Diagrams (State Machine Diagrams):** State diagrams describe how an object or system **transitions** between different states in response to events. These diagrams show **states** and **state transitions**, helping to model the life cycle of an object or a system.

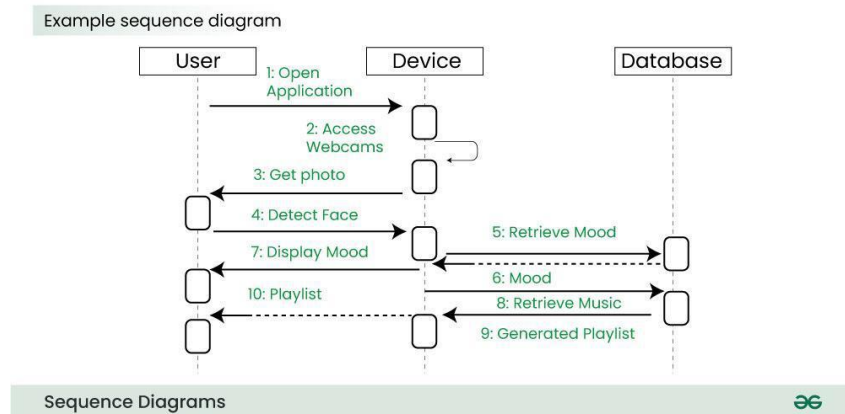
A State Machine Diagram for user verification



State Machine Diagrams | Unified Modeling Language (UML)

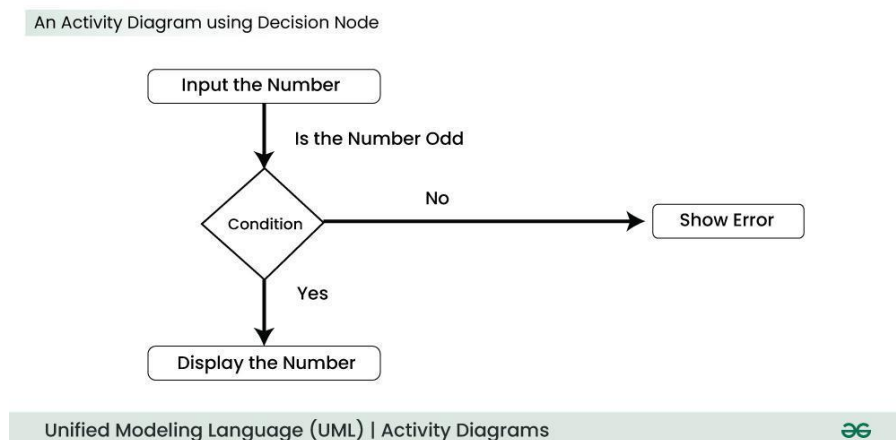
- **Sequence Diagrams:**

Sequence diagrams show the interaction between **objects** or **components** over time. They emphasize the order of messages or method calls and their sequencing in response to external events.



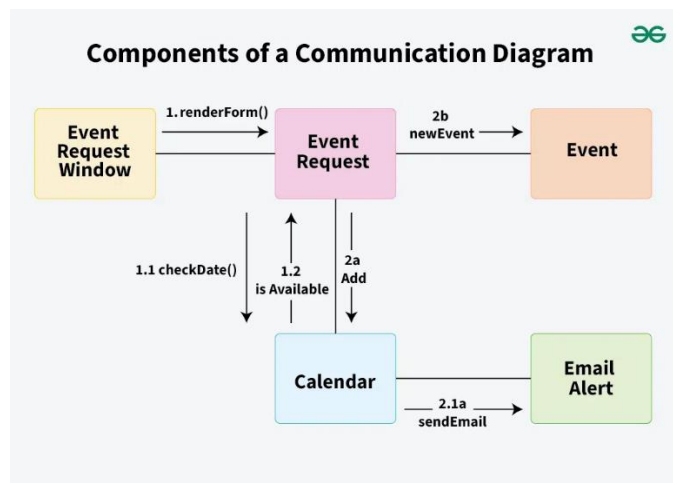
- **Activity Diagrams:**

Activity diagrams represent workflows or processes in the system. They show how various **activities** or **actions** are performed in sequence or in parallel, representing the flow of control and data.



- **Communication Diagrams (Collaboration Diagrams):**

These diagrams focus on the communication between different objects or components, showing how they interact and exchange messages during the execution of a system.



3. Benefits of Dynamic Models:

- Help in **testing** and **simulation** of system behaviour before actual implementation.
- Allow engineers to understand the system's **reaction** to various inputs and states.
- Facilitate the identification of potential problems in system behaviour, such as **race conditions**, **deadlocks**, or **bottlenecks**.

State and Activity Models

State and activity models are dynamic models used to represent **behaviour** over time. They help describe how the system changes its state in response to events or actions and how different processes are executed within the system. **Key Concepts:**

- **State Models:** Focus on the **states** that an object or system can occupy during its life cycle. They describe **transitions** between states triggered by specific **events** or **conditions**. State models help capture the dynamic aspects of a system, such as the transition of a software module between various modes or conditions.
- **Activity Models:** Show the flow of control and data within a system as a sequence of **activities** or **actions**. These models are concerned with how **tasks** or **functions** are performed, including conditions that trigger the flow of one activity to the next.

1. State Models or State Diagram (State Machine):

- Represents the various **states** an object or component can be in during its life cycle, and the **events** or **conditions** that cause transitions between states. These models show both the **current state** and **next state** in response to inputs.
- Example: A ticket booking system may have states such as "Available," "Booked," "Cancelled," etc., with transitions triggered by actions like a user booking a ticket or cancelling the booking.

2. Activity Models or Activity Diagram:

- Visualizes the workflow or sequence of actions in the system. It represents tasks or activities, the **flow of control**, and **decisions** made during the process. Activity diagrams can show both **sequential** and **parallel** activities.
- Example: In an e-commerce application, an activity diagram might illustrate the steps in the order processing workflow—such as "Add to Cart," "Checkout," "Payment," "Shipping," etc.

3. Benefits of State and Activity Models:

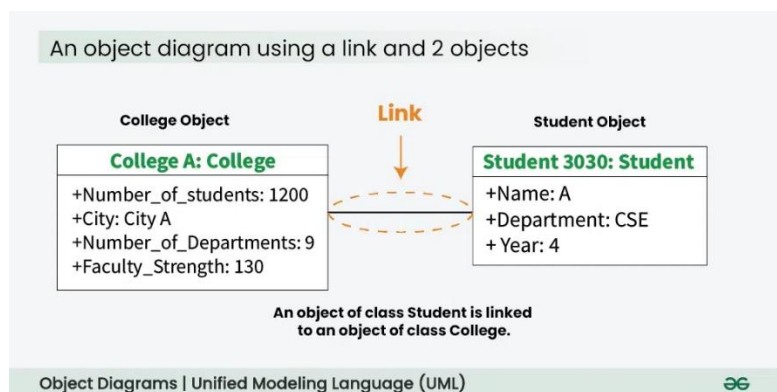
- Provide clarity on how a system behaves in different states or conditions.
- Help in **visualizing workflows** and decision points.
- Allow for the identification of bottlenecks or errors in the flow or state transitions.
- Useful in **test case design** and validating that the system behaves as expected.

Object-Oriented Modelling (Classes, Objects, and Messages)

Object-Oriented Modelling (OOM) is used to represent a system's structure and behaviour using the fundamental concepts of **objects** and **classes**. In OOM, the system is divided into **objects** that interact with each other to perform tasks and process information. **Key Concepts:**

• Classes and Objects:

- A **class** is a blueprint or template for creating objects. It defines the **attributes** (data) and **methods** (functions) that an object of that class will have. An **object** is an instance of a class, representing a specific entity in the system with actual values for its attributes.
- Example: A "Car" class may have attributes like **colour**, **make**, **model**, and **methods** like **drive ()** or **stop()**. An object of the "Car" class could be a specific car with a set colour, make, and model.



- **Messages (Method Calls):**
 - Objects interact with one another by sending **messages** (method calls). When an object needs to perform an action or retrieve information from another object, it sends a message, often invoking a method defined in the receiving object's class.
 - Example: A "Car" object may send a message to a "Driver" object to call the method **drive ()** when the car is ready to move.
- **Encapsulation:** The concept of **encapsulation** in OOM refers to hiding the internal details of an object (e.g., its state and behaviour) while exposing only the **necessary interface** (e.g., public methods) for interacting with the object.
- **Inheritance and Polymorphism:**
 - **Inheritance** allows one class to inherit the properties and behaviours of another class, enabling reusability and extension of functionality. **Polymorphism** allows objects of different classes to be treated as objects of a common superclass, enabling flexibility in handling different types of objects.

Benefits of Object-Oriented Modelling:

- Promotes **reusability** and **modularity** by defining components as objects.
- **Encapsulation** improves system security and reduces complexity by hiding internal implementation details.
- Supports **inheritance** and **polymorphism**, which make it easier to extend the system and accommodate new requirements or changes.
- Makes it easier to model real-world entities and their relationships, simplifying the design process.